XDHDT_MyTeam Home

Capstone Project Document

Group Members	
Supervisor	Ths. Nguyễn Văn Chiến
Capstone Project code	SP26SE025

– Ho Chi Minh, December, 2025 –

I. Project Introduction

1. Overview

1.1. Project Information

- Project name: System for Retinal Vascular Health Screening
- Project code: SP26SE025
- Group name:
- Software Type: Web application

1.2. Project Team

Full name	Role	Email	Phone
Nguyễn Văn Chiến	Supervison	chiennv@ut.edu.vn	
Cao Xuân Dự	Leader	pherocaodu@gmail.com	
Nguyễn Việt Dũng	Member	dungnv5207@ut.edu.vn	
Nông Hùng Phi	Member	nongphi040105@gmail.com	
Nguyễn Văn Liên	Member	lienmeolun@gmail.com	
Đào Duy Vũ	Member	duyvudao79@gmail.com	

2. Product Background

2.1. The Problem and Market Gap

The increasing global focus on preventive and personalized healthcare necessitates the early detection of systemic health risks. However, a significant gap exists: many serious conditions, particularly those related to **cardiovascular, diabetic, and neurological health**, often progress asymptomatically and are discovered too late. Furthermore, access to comprehensive, non-invasive, and regular screening methods in regions like Vietnam is limited due to **high costs, complex procedures, and a lack of specialized personnel** at the local level. This absence of a fast, accurate, and widely deployable screening tool creates a critical public health challenge.

2.2. The Solution: Leveraging Retinal Analysis with AI (AURA)

The Comprehensive AI Understanding Retinal Analysis (**AURA**) system is proposed as an integrated, cloud-based solution. It addresses the access and detection challenges by transforming the routine eye exam into a powerful systemic health screening tool. The system leverages the eye as a "window" to microvascular health,using AI to analyze retinal vessel morphology and detect subtle vascular abnormalities. This analysis allows AURA to **estimate risks of hypertension, diabetes complications, and stroke** at an early stage.

2.3. Core Value Proposition and Strategic Fit

AURA's core value lies in delivering a **scalable, ethical, and data-driven Clinical Decision Support (CDS) tool** that assists—not replaces—physicians. By connecting to existing fundus cameras and utilizing a horizontally scalable cloud architecture (NFR-7), AURA significantly expands access to advanced vascular screening for community clinics. The system is designed for quick performance (NFR-1: 10-20 seconds) and provides **visually explainable outputs (NFR-22)**, ensuring doctors can validate AI findings, thereby improving patient outcomes and reducing diagnostic barriers with speed and confidence.

3. ExistSystem

The development of the Comprehensive AI Understanding Retinal Analysis (AURA) system is necessitated by critical gaps in current healthcare delivery methods, particularly in Vietnam and other underserved regions. Existing solutions often fall short in providing a scalable, non-invasive, and comprehensive assessment of systemic health risks.

We categorize the current landscape into three primary groups, highlighting their limitations that AURA seeks to address:

3.1.Manual Retinal Diagnosis and Specialist Assessment

System/Method: Fundus Camera or OCT imaging followed by manual assessment and diagnosis by an Ophthalmologist or Retinal Specialist.

Description: This is the traditional standard of care where medical specialists visually analyze retinal images to identify signs of specific diseases (e.g., Diabetic Retinopathy, Glaucoma).

Pros: High diagnostic accuracy when performed by highly trained personnel; provides a definitive clinical diagnosis.

Limitations (Addressed by AURA):

+ **High Barrier to Access:** Requires the presence of highly specialized personnel, which is scarce and expensive, particularly in community clinics and mid-tier hospitals (as noted in the project context).

+ Inconsistent Systemic Screening: While specialists can observe changes indicative of systemic health, there is no standardized, fast protocol for using retinal morphology to **quantify** risks of stroke, hypertension, and diabetes complications—a key objective of AURA.

+ Scalability Issue: The process is time-consuming, hindering the ability to perform high-volume, regular, and preventative screenings.

3.2. Existing AI-based Retinal Screening Tools

System/Method: Commercial AI diagnostic tools (e.g., certain FDA-approved systems) or proprietary software predominantly focusing on screening for a specific, single disease.

Description: These tools automate the detection of defined pathologies, most commonly **Diabetic Retinopathy (DR)**, by analyzing retinal images.

Pros: Fast detection; reduces the workload for specialists in high-volume settings.

Limitations (Addressed by AURA):

+ **Narrow Focus:** Most existing AI systems are disease-specific (e.g., only DR). They typically **do not** offer the **Comprehensive AI Understanding Retinal Analysis (AURA)** required to estimate the risk of hypertension, stroke, or general cardiovascular health by analyzing overall retinal vessel morphology.

+ Lack of Explainability: Many commercial AI solutions function as "black boxes," providing a result without transparent, visually explainable findings (e.g., heatmaps, annotated vessels). AURA emphasizes **Ethical, Transparent, and Secure AI Use** ([NFR-22]), which is crucial for physician validation and adoption.

+ Cost of Implementation: Licensing and integration costs for proprietary solutions often remain prohibitively high for the community clinics and grassroots-level organizations targeted by AURA.

3.3. General Cardiovascular Risk Assessment Tools

System/Method: Traditional risk calculators (e.g., Framingham Risk Score, ASCVD Risk Estimator) and non-imaging health devices (e.g., blood pressure monitors, cholesterol tests).

Description: These tools assess risk based on demographic data, lifestyle, and biochemical markers (blood tests).

Pros: Widely accepted and simple to administer.

Limitations (Addressed by AURA):

+ **Not Non-Invasive:** Requires blood draws, which can be invasive, or reliance on self-reported/measured data.

+ **Lack of Direct Visualization:** They do not utilize the direct, objective evidence offered by retinal vessels, which serve as a **"window" to systemic health** and directly reflect microvascular damage.

4.Business Opportunity

The AURA project is strategically positioned to capitalize on major trends in healthcare technology and address critical market deficiencies, particularly in regions facing challenges with specialized medical access, like Vietnam.

4.1. The Global Shift to Preventive Medicine

Market Trend: Global healthcare is rapidly moving from reactive treatment of disease towards proactive prevention and early detection. Cardiovascular, diabetic, and neurological conditions are major burdens on healthcare systems worldwide.

AURA's Value Proposition: AURA offers a solution for **proactive prevention** by transforming routine eye exams into a comprehensive, non-invasive systemic health screening. Early risk detection, enabled by AURA, allows for timely intervention, significantly reducing long-term healthcare costs and enhancing patient quality of life.

4.2. Addressing the Access and Cost Barrier

The Problem: As highlighted in the context, many individuals, especially in community and rural settings, face significant barriers in accessing comprehensive and regular screening due to the high cost of specialized equipment and the lack of trained specialists.

The Opportunity:

+ **Market Expansion:** AURA utilizes a **cloud-based analysis model**, connecting to existing, widely available fundus cameras. This makes advanced vascular screening accessible to a large, underserved market segment, including **community clinics and mid-tier hospitals**, that cannot afford full diagnostic infrastructure.

+ **Cost Efficiency:** By operating as a Clinical Decision Support (CDS) tool via a SaaS (Software as a Service) model, AURA minimizes the capital investment required by healthcare facilities, providing an affordable, pay-per-use alternative.

4.3. Competitive Advantage of Comprehensiveness and AI

Systemic Comprehensive View: While existing AI competitors often focus narrowly on a single disease (e.g., Diabetic Retinopathy), AURA leverages the retina as a **"window to systemic health"** to assess multiple risks simultaneously: hypertension, stroke, and diabetes complications. This comprehensive risk profile is a unique selling point.

Trusted Clinical Decision Support (CDS): AURA is explicitly designed as a CDS tool that assists—not replaces—physicians. Its output is **interpretable and visually explainable** ([NFR-22]), which builds trust and confidence among medical professionals, facilitating faster adoption and integration into clinical workflows.

5. Software Product Vision

The Software Product Vision defines the long-term goal, core philosophy, and strategic direction for the Comprehensive AI Understanding Retinal Analysis (AURA) system. It outlines what AURA will become and the impact it aims to have on preventive healthcare.

5.1. Core Vision Statement

AURA's vision is to be the **leading AI-powered Clinical Decision Support (CDS) platform** that utilizes non-invasive retinal imaging to **quantify and detect early systemic health risks** (cardiovascular, diabetic, and neurological) at the point of care, thereby enhancing quality of life and significantly reducing long-term healthcare burdens.

5.2. Strategic Objectives

Based on the proposed solutions and the system's ethical foundation, AURA aims to achieve the following long-term strategic objectives:

Area	Vision Description
Public Health & Access	**Democratization of Screening:** To become the default, go-to screening tool, successfully deployed in **every community clinic and mid-tier hospital** in the target market, thereby ensuring advanced preventive care is accessible to underserved populations.
Clinical Support	**Indispensable Clinical Aid:** To be recognized as the most trusted CDS tool by doctors. AURA will empower clinicians by providing **fast, validated, and interpretable risk assessments**, allowing them to confidently make rapid decisions and confirm initial diagnoses.

AI Technology & Data	**Continuous Learning Leader:** To build a robust **data collaboration platform** that continuously refines the AI model through real-world feedback from clinics and research institutions, maintaining the highest standards for accuracy, traceability ([NFR-23]), and explainability ([NFR-22]) in retinal vascular analysis.
Ethics & Compliance	**Gold Standard for Medical AI:** To set the highest benchmark for responsible AI adoption in medicine. The system will strictly adhere to medical data protection standards ([NFR-10]), ensuring data privacy, anonymization, and maintaining **transparency** in all AI outputs.

5.3.Future Product Roadmap (Long-term Evolution)

To ensure longevity and market relevance, the vision includes expanding AURA's capabilities:

+ **Advanced Imaging Support:** Integrate analysis for **OCT (Optical Coherence Tomography)** images to provide a deeper understanding of neuronal and sub-surface vascular structures, expanding the scope beyond standard fundus imaging.

+ **Mobile Engagement:** Develop a **Flutter-based mobile application** (as an option) to enhance user engagement, allowing patients to manage their profiles, receive health warnings, and utilize the in-app chat feature to communicate with their assigned doctors ([FR-10]).

+ **Interoperability:** Achieve seamless integration with widely used **Electronic Health Record (EHR) and Hospital Information Systems (HIS)** via standard API endpoints, ensuring AURA becomes a natural part of the modern clinical workflow.

6. Project Scope & Limitations

This section clearly defines the boundaries of the System for Retinal Vascular Health Screening (AURA/SP26SE025) project, outlining what will be included in the development effort and what will be excluded or limited to ensure project feasibility and manage expectations.

User Functional Requirements:
[FR-1] Register and log in using email, Google account, or social authentication.
[FR-2] Upload single or multiple retinal (Fundus or OCT) images for analysis.
[FR-3] View AI-generated diagnostic results and risk levels.
[FR-4] Visualize annotated images showing affected vascular areas.
[FR-5] Receive automated health recommendations or warnings.
[FR-6] Access personal analysis history and previous reports.
[FR-7] Download or export diagnostic reports (PDF/CSV).
[FR-8] Manage and update personal profile and medical information.
[FR-9] Receive notifications when AI results are ready.
[FR-10] Communicate with the assigned doctor via in-app messaging.

[FR-11] Purchase or renew analysis service packages.
[FR-12] View payment history and remaining analysis credits.

Doctor Functional Requirements:
[FR-13] Log in and manage assigned patient profiles.
[FR-14] Review AI analysis results and annotations.
[FR-15] Validate or correct AI-generated findings.
[FR-16] Add medical notes, diagnoses, or recommendations.
[FR-17] Access patient history, previous analyses, and trend data.
[FR-18] Filter or search patients by ID, name, or risk level.
[FR-19] Provide feedback to improve AI accuracy and model retraining.
[FR-20] Communicate with users (patients) through consultation chat.
[FR-21] View performance summaries or analysis statistics.

Clinic Functional Requirements:
[FR-22] Register clinic accounts and verify organization identity.
[FR-23] Manage multiple doctor and user (patient) accounts.
[FR-24] Upload and submit bulk retinal images for AI analysis.
[FR-25] Monitor all patient analysis reports and aggregated risk data.
[FR-26] Generate clinic-wide reports for screening campaigns.
[FR-27] Track number of images analyzed and package usage.
[FR-28] Purchase or renew clinic-level service packages.
[FR-29] Receive alerts for high-risk patients or abnormal trends.
[FR-30]4.3. Export summarized statistics for clinical research or management.

Admin Functional Requirements:
[FR-31] Manage user, doctor, and clinic accounts (enable, disable, edit).
[FR-32] Define and update user roles and access permissions.
[FR-33] Configure AI parameters, thresholds, and retraining policies.
[FR-34] Manage service packages, pricing, and billing models.
[FR-35] Access global dashboard showing usage, revenue, and AI performance.
[FR-36] View system analytics (image count, risk distribution, error rates).
[FR-37] Handle data compliance, audit logs, and privacy settings.
[FR-38] Approve or suspend clinic registrations.
[FR-39] Manage notification templates and communication policies.

Non-functional requirement:

Performance Requirements:
[NFR-1] AI analysis for a single retinal image must complete within 10–20 seconds, depending on image size and model load.
[NFR-2] The system must support bulk processing (≥100 images per batch) with queued or parallel execution.
[NFR-3] Dashboard and result retrieval should load in <3 seconds under normal network conditions.

Reliability & Availability Requirements:
[NFR-4] System uptime must be ≥ 99% excluding scheduled maintenance.
[NFR-5] AI Engine must fail gracefully; if analysis fails, the system returns a clear error and preserves all uploaded images.
[NFR-6] Data must be backed up automatically at least once daily.

Scalability Requirements:
[NFR-7] The architecture must allow horizontal scaling of AI microservices to handle increased image volume.
[NFR-8] Must support multiple clinics and thousands of users without performance degradation.

Security & Privacy Requirements:
[NFR-9] All patient data must be encrypted at rest and in transit (TLS 1.2+; AES-256).
[NFR-10] The system must comply with medical data protection standards (HIPAA-like practices / local health data laws).
[NFR-11] Sensitive data must be anonymized before being used for AI model retraining.
[NFR-12] Access must follow role-based access control (RBAC) for Admin, Clinic, Doctor, and User.

Usability Requirements:
[NFR-13] Web UI should be accessible and responsive across desktop, tablet, and mobile.
[NFR-14] Clinicians must be able to upload images and view results with no more than 3 clicks.
[NFR-15] Annotated images and risk explanations must be clear and clinically interpretable.

Maintainability Requirements:
[NFR-16] AI models, thresholds, and configurations must be updatable without system downtime.
[NFR-17] Codebase must follow modular architecture:
+AI Core Microservice
+Web Application
+Admin/Clinic/Doctor modules
+Database layer
[NFR-18] Logging, auditing, and error tracking must be centralized.

Interoperability Requirements:
[NFR-19] Must support integration with common retinal fundus cameras (via cloud upload).
[NFR-20] Export formats must include PDF, CSV, and standardized medical formats where possible.
[NFR-21] AI engine must communicate over standard RESTful API endpoints.

Data Quality & Explainability Requirements:
[NFR-22] AI outputs must include explainable elements (heatmaps, annotated vessels, highlighted abnormalities).
[NFR-23] System must track AI version and thresholds attached to each generated report to ensure traceability.

6.1. Project Scope

The core scope of AURA is focused on providing a comprehensive and scalable AI solution for healthcare providers:

Area	Scope Description
Core Product Goal	Develop and deploy an AI-powered Clinical Decision Support (CDS) Platform that analyzes retinal images to assess, quantify, and categorize the risk of systemic diseases (cardiovascular, diabetic, neurological).
User Roles & Functionality	Fully implement the required functionalities (from [FR-1] to [FR-39]) for the four defined user roles: User (Patient), Doctor, Clinic, and Admin.
Deliverables	Develop the three main components: the AI Core Microservice API, the Clinical Decision Support (CDS) Web Application, and the Admin/Management Web Application.
AI Analysis	Deploy AI/Machine Learning algorithms (using Python) capable of vascular analysis, abnormality detection, and generating explainable results (heatmaps, annotated images) ([NFR-22]).
Technology Stack	Utilize the specified technologies: Server (.NET, Docker), Database (PostgreSQL/MongoDB), and Client (React + TypeScript).
Performance & Scalability	Ensure the system meets Non-Functional Requirements (NFRs) regarding performance (AI analysis speed of 10–20 seconds ([NFR-1])) and scalability (supporting bulk processing and thousands of users ([NFR-7], [NFR-8])).

6.2. Project Limitations

The following limitations are established to manage expectations and ensure the project can be completed within the defined timeframe and resources (typical for an academic/capstone project):

Area	Specific Limitation
Diagnostic Role	The system is strictly a Clinical Decision Support (CDS) tool. AI-generated results and risk estimations are suggestions and do not constitute a final medical diagnosis. The Doctor ([FR-15]) retains the sole responsibility for final clinical judgment.
Regulatory Certification	The project commits to adopting best practices for health data security (e.g., data encryption [NFR-9], anonymization [NFR-11]). However, achieving full, official certification from national or international medical regulatory bodies (such as the FDA or local Ministry of Health) is out of scope for this academic project.

Hardware Integration	The system will prioritize supporting standard image formats (Fundus/OCT) via web/cloud upload mechanisms ([FR-2], [NFR-19]). Developing complex, automated, direct hardware integration protocols with all different types of fundus cameras is excluded from the core scope.
Client Development	The main deliverable is the Web Application. The development of the Mobile Application (using Flutter), listed as (optional) in the proposal, will be excluded from the core development scope unless sufficient resources and time remain after the core Web product is finalized.
AI Accuracy	The performance and accuracy of the AI model will be constrained by the size, quality, and diversity of the training datasets available for use during the project development timeline.
Payment System	The functionality for purchasing/renewing service packages ([FR-11]) will be integrated with a testing/sandbox payment gateway or simulated using internal account management features, not a production-ready payment gateway during the initial phase.

II. Project Management Plan

1. Overview

1.1 Scope & Estimation

#	WBS	Complexity	Est. Effort (man - hour)
1	**Initiating**		24
1.1	Define project scope	Medium	13
1.2	Collect requirements	Medium	11
2	**Planning**		14
2.1	Create kick-off meeting	Medium	8
2.2	Create plan document	Medium	6
3	**Executing**		
3.1	**Analysis**		36
3.1.1	Analysis requirements	Medium	23
3.1.2	Feasibility Analysis	Complex	13
3.2	**Design**		60
3.2.1	UI/UX Design	Medium	12

3.2.2	System & Database Design (Architecture, ERD)	Medium	20
3.2.3	Database Schema Finalization	Medium	8
3.2.4	SRS Finalization	Complex	20
3.3	**Core Implementation**		48
3.3.1	AI Model Development & Initial Training	High	24
3.3.2	Core API Development	Medium	10
3.3.3	AI Core Integration API (RESTful endpoint)	Medium	8
3.3.4	Payment/Subscription API	Low	6
3.4	**Web Application**		80
3.4.1	User Module Development (Frontend)	Medium	25
3.4.2	Doctor & Clinic Modules Development (Frontend)	High	40
3.4.3	Admin Dashboard Development (Frontend)	Medium	15
3.5	**Testing**		75
3.5.1	Unit & Integration Testing	Medium	20
3.5.2	System & Performance Testing (NFR-1, NFR-3)	High	30
3.5.3	Packaging & Deployment (Dockerize, VPS Setup)	Medium	25
3.6	**Monitoring and Controlling**		40
3.6.1	Control the process	Complex	20
3.6.2	Track performance a quality	Complex	20
3.7	**Closing**		20
3.7.1	Report	Simple	20

1.2 Project objectives

Timelines: The project must be finished before 18 févr. 2026

Defect Distributions:

#	Testing Stage	Test Coverage (Target)	No. of Defects	% of Defect	Notes

1	Reviewing	**Docs:** 100% SRS & Use Cases **Design:** 100% DB Schema & Wireframes	<5	1%	Review all specification documents, database schema (ERD), and sample interfaces for the four roles (User, Doctor, Clinic, Admin).
2	Unit Test	**BE (.NET):** > 85% Services (Auth, Report) **AI Core:** > 90% Pre-processing funcs **FE (React):** > 70% Core Components	<15	5%	Focus on business process services, AI image processing functions, and key components (Upload, Dashboard).
3	Integration Test	**API:** 100% RESTful Endpoints **Flow:** 100% Web -> AI Communication	<5	5%	Test all defined API endpoints and the data flow between the Web Server and the AI Service.
4	System Test	**Function:** 100% FRs [FR-1 -> FR-39] **Performance:** Load < 3s, AI < 20s	<12	20%	Test all 39 functional requirements and key performance metrics (response time, load capacity).
5	Acceptance Test	**UAT:** 100% "Happy Path" Scenarios **UX:** Doctor Validation	<6	10%	Test the main workflow (Happy Path) and confirm the comprehensibility of the AI results from the doctor's perspective.

1.3 Project Ricks

#	Risk Description	Impact	Possibility	Response Plans

1	Low AI accuracy: The AI model does not achieve the desired accuracy or produces too many false positives/false negatives in detecting vascular abnormalities.	**Critical:** The system fails to serve as a reliable Clinical Decision Support tool. Doctors will reject the results, leading to a failure in User Acceptance Testing (UAT) and potential rejection of the project at the Final Defense.	High	• Use transfer learning with pre-trained medical models. • Implement data augmentation. • Consult with doctors for ground-truth labeling.
2	Training data shortage: Insufficient high-quality retinal images (Fundus/OCT) or unbalanced data (too few pathological images) are available to train the AI.	**High:** The AI model cannot be trained effectively, resulting in poor performance. The core feature "Vascular Analysis" becomes dysfunctional, rendering the product incomplete.	Medium	• Use open datasets (DRIVE, STARE, Kaggle). • Apply oversampling techniques. • Seek partnerships with local clinics early.
3	System Integration Failure: Communication between the .NET Backend and the Python AI Service fails (timeout, incorrect data format) when processing large amounts of traffic or high-resolution images.	**High:** The end-to-end workflow is broken. Users can upload images but cannot receive reports. The system is functionally useless for the end-user.	Medium	• Define strict API specifications (Swagger). • Use asynchronous processing (Message Queue). • Dockerize all services to ensure environment consistency.
4	Performance Issues: Image analysis time exceeds 20 seconds [NFR-1] due to limited server resources or suboptimal algorithms.	**Medium:** Violation of Non-Functional Requirement [NFR-1]. It disrupts the clinical workflow, leading to poor User Experience (UX) and point deductions during the Performance Grading.	Medium	• Resize/normalize images before processing. • Optimize model inference (e.g., using ONNX). • Utilize GPU-accelerated cloud services if budget permits.

5	Medical Data Privacy Breach: Patient data or retinal images are leaked due to poor security and violations of ethical standards.	**Critical:** Severe violation of medical ethics and compliance standards. This could lead to immediate disqualification of the project and legal issues regarding data protection.	Low	• Encrypt all data (AES-256). • Anonymize data before AI training. • Implement strict RBAC.
6	Schedule Slippage: Failure to complete complex modules (such as AI Core) within the 12-week deadline due to underestimating the difficulty.	**High:** Failure to deliver the committed Scope. The team may be forced to remove features (de-scope), leading to an incomplete product handover and lower evaluation scores.	High	• Follow Agile/Scrum strictly. • Prioritize MVP features (Core flow) first. • Monitor velocity weekly.
7	Skill Gap: Team members struggle to learn new technologies (Docker, Deploy AI, TypeScript) in a short period of time.Medium	**Medium:** Development velocity slows down significantly in the first 3 weeks. Code quality may suffer, resulting in a higher defect rate during the Integration Test phase.	Medium	• Pair programming (Senior-Junior). • Dedicate Week 1 for R&D and training. • Code reviews to ensure quality.

2. Management Approach

To ensure the AURA project meets its ambitious goals within the 10-week timeframe, the team adopts a structured yet flexible management approach. This approach emphasizes iterative development, continuous feedback, and strict quality control standards to handle the complexity of integrating AI models with a web-based clinical platform.

2.1 Project Process

The project follows the Scrum Agile Framework, utilizing an iterative and incremental development model. This allows the team to develop the AI Core Microservice and Web Application concurrently while adapting to technical challenges and feedback from supervisors.

Process Overview:

Methodology: Agile Scrum.

Sprint Duration: 2 Weeks.

Total Sprints: 6 Sprints.

Key Ceremonies: Sprint Planning, Daily Stand-up, Sprint Review, and Sprint Retrospective.

The development lifecycle is divided into four strategic phases:

Phase 1: Inception

Objective: Define scope, analyze requirements, and establish the technical foundation.

Key Activities:

Finalizing the Software Requirement Specification (SRS).

Designing the Database Schema (PostgreSQL) and System Architecture.

Setting up the development environment (Docker, GitHub Repositories).

Phase 2: Elaboration

Objective: Mitigate key technical risks and build the architectural baseline.

Key Activities:

Developing the initial AI Model for retinal vessel segmentation.

Implementing core Backend APIs (.NET) for Authentication and File Upload.

Establishing the communication pipeline between the Web Server and AI Service.

Phase 3: Construction

Objective: Complete all functional requirements and refine system performance.

Key Activities:

Developing Frontend modules for all 4 roles (User, Doctor, Clinic, Admin).

Integrating the AI analysis results (Heatmaps, Risk Scoring) into the Doctor's Dashboard.

Conducting rigorous Integration Testing to ensure data integrity.

Phase 4: Transition

Objective: Finalize the product for delivery and deployment.

Key Activities:

Performance tuning (ensuring AI analysis < 20s).

User Acceptance Testing (UAT).

Final Deployment to Cloud/VPS and Documentation (User Manuals).

2.2 Quality Management

Quality Management ensures that the AURA system is reliable, maintainable, and meets all defined Functional and Non-Functional Requirements. We adhere to the philosophy of **"Prevention over Inspection"** through the following practices:

2.2.1 Coding Standards & Conventions

To maintain code consistency across the team, strict coding conventions are enforced:

+ **Backend (.NET):** Adhere to Microsoft's C# Coding Conventions (PascalCase for methods/classes, camelCase for local variables).

+ **Frontend (React/TypeScript):** Follow Airbnb Style Guide, enforced via **ESLint** and **Prettier**.

+ **AI Core (Python):** Follow **PEP 8** standards for Python code to ensure readability and maintainability.

2.2.2 Code Review Process

We utilize **GitHub** for source code management with a mandatory review workflow:

+ **Pull Requests (PR):** No code is committed directly to the `main` branch. Developers must create a feature branch and submit a PR.

+ **Peer Review:** Every PR requires approval from at least one other team member. Reviewers check for logic errors, security vulnerabilities, and adherence to coding standards.

+ **Cross-Functional Review:** Frontend and Backend developers review each other's API-related code to ensure seamless integration.

2.2.3 Defect Management

+ **Tracking Tool:** All bugs and issues are logged in **Jira/GitHub Issues**.

+ Severity Levels:

Critical: System crash or data loss (Must fix immediately).

High: Core functionality broken (Fix before next release).

Medium: UI/UX issues or minor logic errors.

Low: Typos or cosmetic suggestions.

2.2.4 Definition of Done (DoD)

A user story or task is considered "Done" only when it meets the following criteria:

+ Code is implemented and pushed to the repository.

+ Code has passed the **Peer Review** process.

+ Unit Tests are written and passed (Green build).

+ Functionality has been verified on the staging environment.

+ No critical or high-severity bugs remain open.

2.3 Training Plan

To ensure that all team members possess the necessary technical skills and domain knowledge required for developing the AURA system, a structured training plan is established at the early stage of the project. The training activities focus on both software engineering technologies and AI-related components, and are scheduled to minimize risks during the implementation phase.

The training plan is mandatory for all team members unless explicitly stated otherwise. Training sessions are conducted through self-study, guided tutorials, internal knowledge sharing, and supervised practice.

Training Area	Participants	When, Duration	Waiver Criteria
Python for AI & Data Processing	All members	Week 1 - 16 Hours	Mandatory
Deep Learning & Medical Image Processing	AI core members	Week 1 - 20 Hours	Mandatory
.NET Backend Development	Backend members	Week 2 - 20 Hours	Mandatory

React & TypeScript Basics	Frontend members	Week 2 - 16 Hours	Mandatory
Database Design (SQL)	All members	Week 2 - 8 Hours	Mandatory
Git & Version Control	All members	Week 1 - 8 Hours	Mandatory

3. Project Deliverables

#	Deliverable	Due Date	Notes
1	Sprint 1	03/12/2025 – 07/12/2025	• Project kickoff meeting • Collect and analyze user requirements • Study problem domain and related systems • Team training on tools and technologies
2	Sprint 2	08/12/2025 – 12/12/2025	• Finalize Software Requirement Specification (SRS) • Define functional and non-functional requirements • Identify system constraints and assumptions • Report 1
3	Sprint 3	13/12/2025 – 17/12/2025	• Design overall system architecture • Design database schema (ERD) • Define AI service workflow
4	Sprint 4	18/12/2025 – 22/12/2025	• Design UI/UX wireframes for User, Doctor, Clinic, and Admin • Review and refine UI design based on feedback
5	Sprint 5	23/12/2025 – 27/12/2025	• Set up base project structure • Configure backend (.NET) and frontend (React + TypeScript) • Configure CI/CD pipeline • Report 2

6	Sprint 6	28/12/2025 – 01/01/2026	• Implement authentication and authorization module • Implement user management module • Role-based access control
7	Sprint 7	02/01/2026 – 06/01/2026	• Implement retinal image upload module • Develop initial AI core processing pipeline • Integrate AI service with backend
8	Sprint 8	07/01/2026 – 11/01/2026	• Implement doctor and clinic management modules • Display AI analysis results and annotations
9	Sprint 9	12/01/2026 – 16/01/2026	• Implement reporting and data visualization features • Optimize system performance
10	Sprint 10	17/01/2026 – 21/01/2026	• System integration testing • Fix bugs and refine system features
11	Sprint 11	22/01/2026 – 27/01/2026	• Conduct system and performance testing • Validate functional and non-functional requirements
12	Sprint 12	28/01/2026 – 04/02/2026	• Final system stabilization • Prepare final report and presentation slides • Report 3

4. Responsibility Assignments

D~Do; R~Review; S~Support; I~Informed; - Omitted

Responsibility	CaoXD	DungNV	PhiNH	LienNV	VuDD
Project Planning & Tracking	D	R	S	S	S
Prepare Project Introduction Document	R	D	S	S	S
Prepare Software Requirement Specification (SRS)	R	D	D	S	S
Prepare Software Design Document (SDD)	D	R	S	D	S

Prepare AI Core Design & Analysis	R	S	D	S	D
Prepare Backend & System Architecture	D	R	S	D	S
Prepare UI/UX Design	R	S	S	D	D
Prepare Software Testing Document	D	R	S	S	D
Prepare Software User Guides	S	D	S	S	R
Prepare Final Report	R	D	S	S	D

5. ProjCommunication

Communication Item	Who/Target	Purpose	When	Type, tool, methods
Daily Stand-up	Internal Team (Devs, PM, BA)	• Update progress (Done/To-Do). • Raise blockers/issues. • Synchronize daily tasks.	Daily	Online Meeting
Weekly Supervisor Meeting	Team & Supervisor	• Report weekly progress. • Demonstrate completed features. • Consult on technical/medical issues.	Weekly	Offline / Hybrid
Sprint Planning	Internal Team	• Define Sprint Goal. • Select items from Product Backlog. • Estimate effort.	Every 2 Weeks	Workshop

Sprint Review & Demo	Team & Supervisor	• Showcase the working product increment. • Receive feedback. • Validate Acceptance Criteria.	Every 2 Weeks	Presentation
Tech Integration Sync	Backend Devs & AI Engineers	• Solve integration issues between .NET API and Python AI Service. • Define Data Contracts.	Ad-hoc	Technical Discussion
General Updates & Alerts	All Members	• Quick Q&A, announcements. • **Urgent:** Critical bug alerts or server downtime.	Anytime	Instant Messaging
Code Review	Developers	• Ensure code quality. • Check logic & security before merging.	Upon Pull Request	Async Review

6. Configuration Management

6.1 Document Management

The project utilizes Atlassian Confluence as the central Knowledge Base to ensure transparency and traceability of requirements.

Documentation & Knowledge Base:

Confluence: Used for creating and storing dynamic documents including the Software Requirement Specification (SRS),API Documentation, and Medical Research Notes. All team members can collaborate in real-time.

6.2 Source Code Management

Source code integrity and collaboration are managed using **Git** and **GitHub**, strictly following the Git Flow branching strategy.

- **Source Code Repository:** Code is hosted on GitHub. The repository is structured to support the microservices architecture (Frontend, Backend, AI Service).
- **Traceability:** GitHub is integrated with Jira. Every commit message must include the Jira Issue Key . This ensures that every line of code can be traced back to a specific functional requirement or bug report.
- **CI/CD (Optional):** GitHub Actions is configured to automatically build and test the code whenever a Pull Request is created.

6.3 Tools & Infrastructures

The project employs a containerized environment to ensure consistency between development and production stages.

- **Project Management:**
  - **Jira Software:** The primary tool for Agile Project Management. It is used to manage Sprints, track User Stories, and log Defects. Jira is linked with Confluence to map Requirements to Tasks.
- **Containerization:**
  - **Docker:** Used to containerize the application services.
    - *Backend:* Dockerfile for .NET Core API.
    - *AI Core:* Dockerfile for Python environment (ensuring exact library versions like PyTorch, OpenCV).
    - *Orchestration:* **Docker Compose** is used to run the entire stack (Web + AI + DB) locally with a single command.
- **Development Tools:**
  - **Visual Studio Code :** For coding.
  - **PostgreSQL & pgAdmin:** For database management.

III. Software Requirement Specification

1 Product Overview

1.1. Product purpose and guiding principle

AURA (Comprehensive AI Understanding Retinal Analysis) is a Clinical Decision Support (CDS) system based on retinal imaging (Fundus/OCT), aiming to:

- Estimate risk level and detect signs of retinal microvascular abnormalities.
- Provide visual explanations (heatmaps/annotated vessels) so clinicians can validate AI findings.
- Standardize the screening workflow: upload → analysis → results → report → history tracking.

Guiding principle: AURA assists clinicians in decision-making and DOES NOT replace the final clinical diagnosis.

1.2. Context summary

In the SRS, we only conclude the high-level needs:

- A fast, non-invasive screening tool that is deployable across multiple care levels.
- Clear, explainable outputs to build clinical trust and adoption (explainability).

1.3. System scope

In-scope capabilities:

- Accounts and role-based access control (RBAC: User/Doctor/Clinic/Admin).
- Single and bulk image upload, with queued analysis job processing.
- AI analysis outputs: risk level + findings + explainable image assets.
- History, reporting (PDF/CSV), and AI version traceability per result.
- Notifications for result-ready/high-risk alerts; User–Doctor consultation chat.
- Service packages/credits and transaction recording (at least "sandbox/simulated" if a real payment gateway is not integrated yet).

Out-of-scope (or limited within the capstone scope):

- Official medical regulatory certification as a commercial-grade product.
- Direct integration with all fundus camera vendors via proprietary protocols (prioritize standard file upload).
- Mobile app is optional (only after core web features are completed).

1.4. User classes and usage goals

- User (Patient): submit images, view results/recommendations, manage profile, track history, chat with doctors, purchase packages.

- Doctor (Clinician): review AI outputs, validate/correct findings, add notes, monitor trends, provide consultation.

- Clinic (Healthcare organization): manage multiple accounts, bulk upload, monitor aggregated risk dashboards, generate campaign reports.

- Admin: manage accounts/roles, configure AI thresholds/policies, monitor system, enforce audit/compliance, manage packages.

1.5. High-level modules

(Mapped to FR/NFR; details appear in Functional/Non-functional Requirements)

- Auth/Profile/RBAC: FR-1, FR-8, NFR-12.
- Imaging & AI Jobs: FR-2..5, NFR-1..2, NFR-5, NFR-7.

- Results/History/Reporting: FR-6..7, NFR-20, NFR-22..23.
- Notification & Messaging: FR-9..10, FR-20, FR-29.
- Billing/Packages: FR-11..12, FR-28.
- Clinic & Admin Ops: FR-22..27, FR-30..39, NFR-9..11, NFR-18.

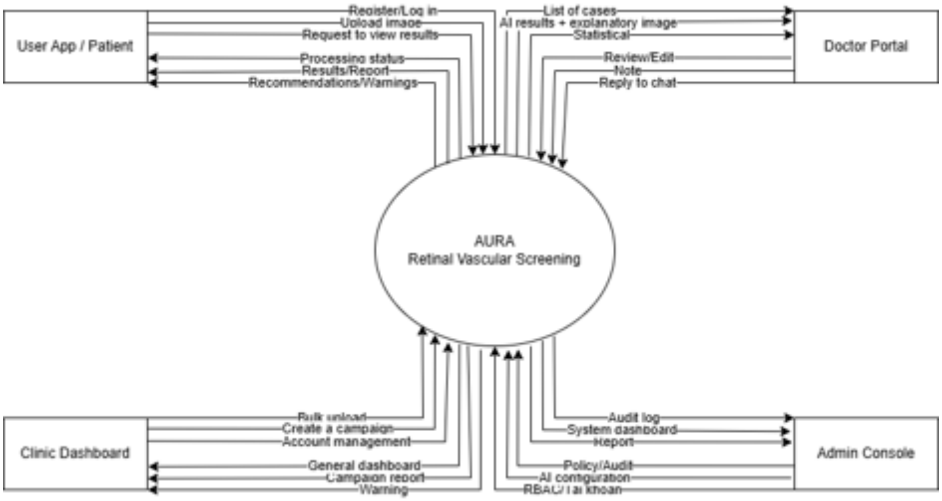
1.6. Operating environment and external dependencies

- Web UI: responsive (NFR-13) and optimized for clinician workflows (NFR-14..15).
- Backend: .NET API, deployed via Docker/VPS/Cloud.
- AI Core: Python microservice, horizontally scalable (NFR-7).
- Storage: DB + Object Storage (images, reports).
- External integrations: OAuth/SSO, notification gateway, payment provider (can be sandbox).
- Communication: RESTful APIs, TLS 1.2+ (NFR-21, NFR-9).

1.7. Constraints and assumptions

- CDS only: clinicians remain responsible for the final clinical decision.
- Image quality impacts reliability; failures must be returned clearly and safely (NFR-5).
- Security & privacy: encryption, RBAC, auditing; retraining data must be anonymized (NFR-9..12, NFR-11, FR-37).
- Traceability: each report must record the AI version/thresholds used (NFR-23).

1.8. Context Diagram



2. User Requirements

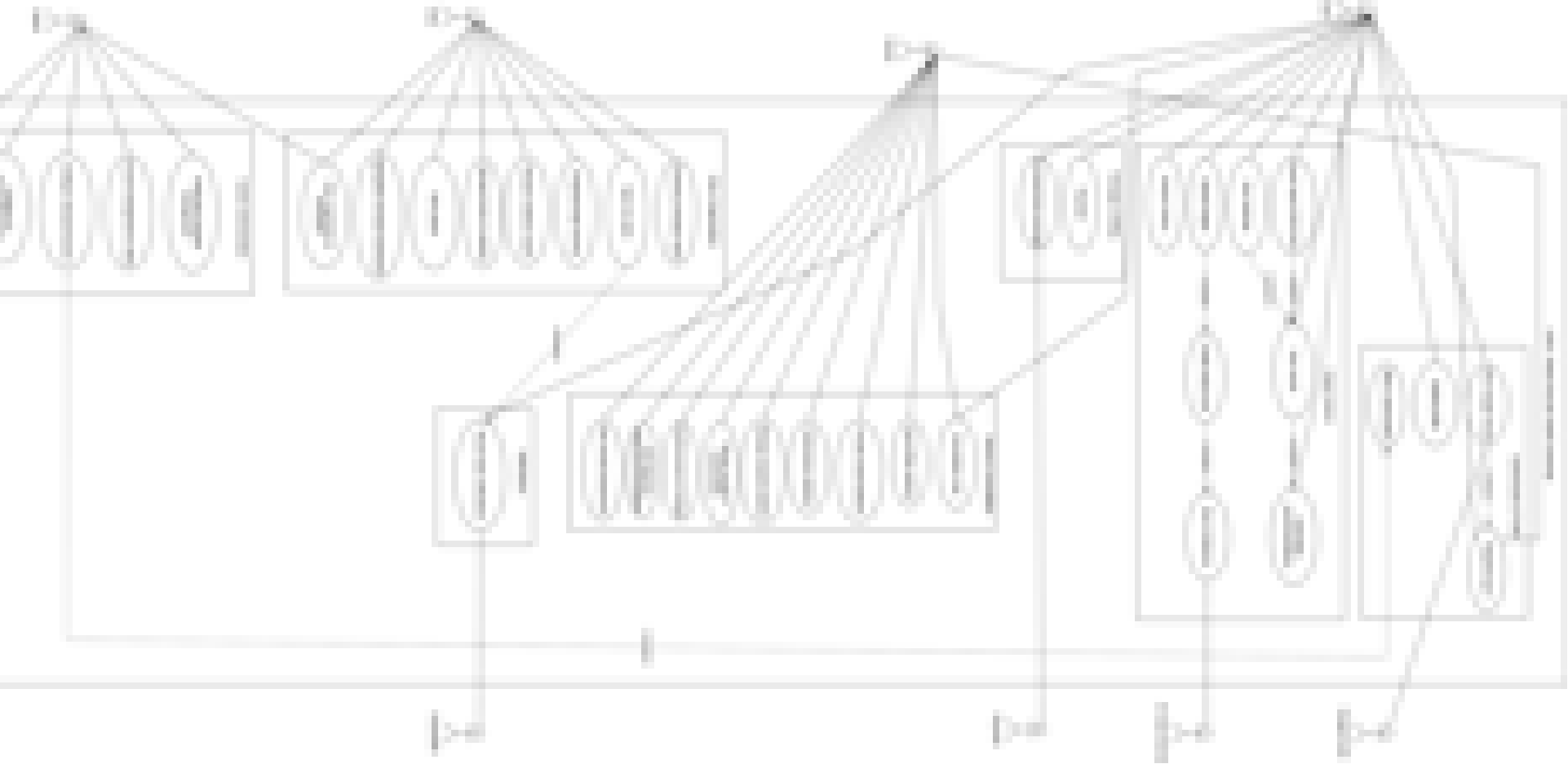
2.1 Actors

#	Actor	Description

1	Admin	Manage the entire AURA platform: manage accounts/roles/permissions (RBAC), monitor activity, configure AI policies and thresholds, manage packages/pricing (if applicable), track audit logs, and manually secure data.
2	Doctor	Doctors/clinicians use the doctor portal to view case lists, see AI results and explanatory images, confirm/correct findings, add clinical notes, consult with patients, and track statistics/trends.
3	User/ Patient	End users: register/log in, upload retinal images (Fundus/OCT), monitor processing status, view results/reports and recommendations, manage profiles & history, receive notifications and (if applicable) purchase/renew service packages.
4	Clinic	Healthcare facility representatives manage multiple accounts/units, conduct bulk uploads (screening campaigns), monitor the aggregated dashboard, generate campaign reports, and receive alerts for high-risk cases.

2.2 Use Case

2.2.1 Diagrams



2.2.2 Descriptions

ID	Use Case	Actors	Use Case Descriptions
01	Sign up / Sign in	User, Doctor, Clinic, Admin	Create an account and/or authenticate to access AURA. The system validates credentials and establishes a role-based session.
02	Google/Social SSO	User, Doctor, Clinic, Admin, Google/OAuth (SSO)	Authenticate via Google/Social using OAuth/OIDC; AURA links returned identity claims to an AURA account
03	Update profile	User	View and update personal profile and relevant medical information used for screening context and history tracking
04	RBAC authorization	Admin	Define and enforce role-based access control rules to protect functions/data across the platform
05	Upload images	User	Upload retinal images (fundus/OCT). The system performs basic validation and stores the uploaded files for processing
06	Create analysis job	User, Clinic	Create one or more queued analysis jobs from uploaded images (single or batch) to enable asynchronous processing
07	Run AI inference	AI Core Service	Perform inference for an analysis job and return risk scores, findings, and references to explainability assets
08	View results	User, Doctor	View analysis results including risk level and key findings for each screening run
09	View heatmap	User, Doctor	View explainability visuals (heatmap/annotations) associated with results to support transparency and review
10	Automated recommendations	User, Doctor	Generate and present recommendations/alerts based on AI outputs and configured thresholds (CDS support only)

11	Analysis history	User, Doctor	View a historical timeline of analyses for tracking and comparison over time
12	Export report	User, Doctor	Export results to PDF/CSV including model/version/threshold traceability for sharing and reporting
13	Manage assigned patients	Doctor	View and manage the list of patients/cases assigned to the doctor (by clinic/campaign/system rules)
14	Review results & explainability	Doctor	Review AI results and explainability assets before confirming a clinical conclusion
15	Confirm/correct AI findings	Doctor	Confirm or correct AI-suggested findings to reflect clinical judgment
16	Add diagnosis notes/recommendations	Doctor	Record clinical notes, conclusions, and recommended next steps for a case
17	View trends/comparisons	Doctor	Analyze trends and compare longitudinal results to assess progression
18	Search/filter patients	Doctor	Search and filter cases by criteria (date range, risk, clinic, review status, etc.)
19	Provide model feedback	Doctor	Submit structured feedback/labels/notes to support model improvement under data governance policies
20	Consultation chat	User, Doctor	Exchange messages for consultation and follow-up, with conversation history and notifications
21	Performance statistics	Doctor	View workload and performance statistics (case counts, review rate, risk distribution, etc.)
22	Clinic registration & verification	Clinic, Admin	Clinic registers and submits verification info; Admin reviews/approves to activate organization capabilities
23	Manage doctors & patients	Clinic	Manage organization members (doctors/patients), internal permissions, and assignments within the clinic

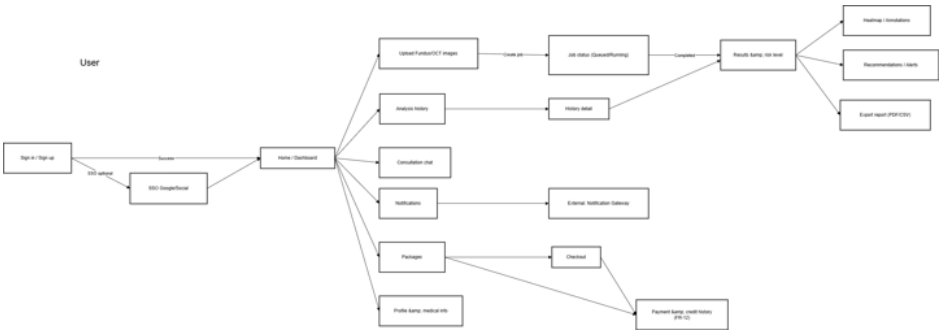
24	Bulk upload	Clinic	Upload image batches for mass screening; create and monitor batch jobs and processing status
25	Aggregated risk/report dashboard	Clinic	View aggregated dashboards summarizing risk distribution and screening outcomes across the organization/campaigns
26	Screening campaign report	Clinic	Generate and export screening campaign reports (summary, flagged cases, statistics)
27	Usage/package tracking	Clinic	Track service usage, credits, and package status for operational management
28	High-risk alerts	Clinic	View/receive alerts for high-risk cases to prioritize follow-up and outreach
29	Research statistics export	Clinic	Export aggregated/anonymized statistics for research and internal analysis, per privacy policy
30	Purchase/renew package	User, Clinic, Payment System	Purchase or renew packages via VNPay/Stripe; handle confirmations/webhooks to activate packages or add credits
31	Payment & credit history	User, Clinic	View transaction records, payment status, credit balance, and package validity period
32	Result-ready notifications	User, Email/SMS/Push Gateway	Send notifications when results are ready or alerts occur and track delivery status via the gateway
33	Account management & status	Admin	Manage user accounts (list, approve, lock/unlock, status changes) and support account incidents
34	Configure roles/permissions	Admin	Configure role/permission settings used by RBAC and apply policy changes
35	Configure AI thresholds/policies	Admin	Configure inference thresholds/policies and model-related settings used for classification and reporting
36	Manage packages/pricing/billing	Admin	Define packages, pricing, billing rules, and credit policies

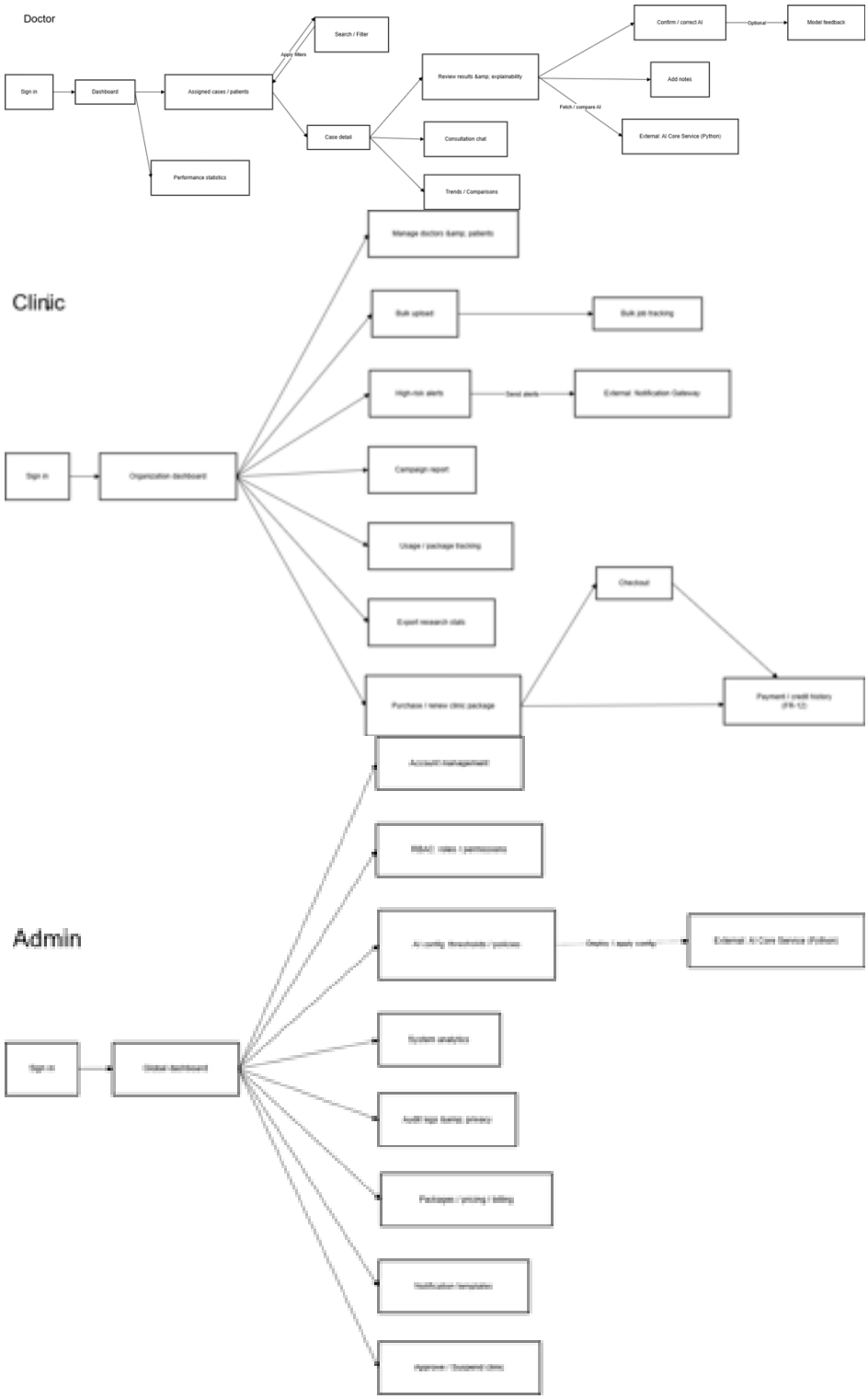
37	Global dashboard: usage/revenue/AI	Admin	View global dashboards for usage, transactions, AI performance, and operational KPIs
38	Audit logs & privacy settings	Admin	Monitor audit logs, manage privacy/security settings, and ensure traceability and compliance
39	Notification template management	Admin	Create and manage notification templates (email/SMS/push) and event-based content

3 Functional Requirements

3.1 System Functional Overview

3.1.1 Screens Flow





3.1.2 Screen Descriptions

(A) User Screens

No	Screen Name	Description
1	Sign In / Sign Up	Allows users to authenticate or register an account, supporting standard login and SSO.
2	Home / Dashboard	Main dashboard providing access to image upload, analysis history, consultation chat, and notifications.
3	Upload Fundus/OCT Images	Enables users to upload Fundus or OCT images to create an AI analysis job.
4	Job Status	Displays the processing status of uploaded images (Queued, Running, Completed).
5	Results & Risk Level	Shows AI analysis results and the detected risk level.
6	Heatmap / Annotations	Visualizes AI-highlighted regions of interest on medical images.
7	Recommendations / Alerts	Provides medical recommendations and alerts based on AI results.
8	Analysis History	Displays a list of all previous analysis records.
9	History Detail	Shows detailed information for a selected analysis record.
10	Consultation Chat	Allows users to communicate with doctors for medical consultation.
11	Notifications	Displays system notifications and alerts.
12	Packages	Shows available service packages for users.
13	Checkout	Allows users to complete package payment.
14	Profile & Medical Information	Enables users to manage personal and medical profile information.
15	Export Report	Allows users to export analysis reports in PDF or CSV format.

(B) Doctor Screens

No	Screen Name	Description
1	Sign In	Allows doctors to log in to the system.
2	Dashboard	Displays an overview of assigned patients and cases.
3	Assigned Cases / Patients	Lists all cases and patients assigned to the doctor.
4	Search / Filter	Enables searching and filtering of assigned cases.
5	Case Detail	Displays detailed information of a specific case.

6	Review Results & Explainability	Allows doctors to review AI results with explainability information.
7	Confirm / Correct AI Results	Enables doctors to confirm or correct AI-generated results.
8	Add Medical Notes	Allows doctors to add clinical notes to a case.
9	Consultation Chat	Enables real-time communication with patients.
10	Trends / Comparisons	Displays trends and comparison of patient results over time.
11	Performance Statistics	Shows performance and workload statistics of doctors.
12	Model Feedback (Optional)	Allows doctors to provide feedback to improve AI models.

(C) Clinic Screens

No	Screen Name	Description
1	Sign In	Allows clinic staff to authenticate into the system.
2	Organization Dashboard	Displays an overview of clinic operations and statistics.
3	Manage Doctors & Patients	Enables clinics to manage doctors and patient accounts.
4	Bulk Upload	Allows bulk upload of medical images for multiple patients.
5	Bulk Job Tracking	Tracks processing status of bulk analysis jobs.
6	High-Risk Alerts	Displays alerts for high-risk patient cases.
7	Campaign Report	Provides reports for screening or medical campaigns.
8	Usage / Package Tracking	Tracks package usage and service consumption.
9	Export Research Statistics	Allows exporting anonymized data for research purposes.
10	Purchase / Renew Package	Enables clinics to purchase or renew service packages.
11	Checkout	Handles payment processing for clinic packages.
12	Payment / Credit History	Displays clinic payment and credit usage history.

(D) Admin Screens

No	Screen Name	Description
1	Sign In	Allows administrators to log in to the system.
2	Global Dashboard	Displays system-wide statistics and operational metrics.
3	Account Management	Manages all user accounts in the system.

4	RBAC Management	Controls roles and permissions across the platform.
5	AI Configuration	Manages AI thresholds, policies, and configuration settings.
6	System Analytics	Provides detailed analytics on system performance and usage.
7	Audit Logs & Privacy	Tracks system activities for security and compliance.
8	Packages / Pricing / Billing	Manages pricing plans, packages, and billing rules.
9	Notification Templates	Manages system notification templates.
10	Approve / Suspend Clinic	Approves, suspends, or manages clinic accounts.

3.1.3 Screen Authorization

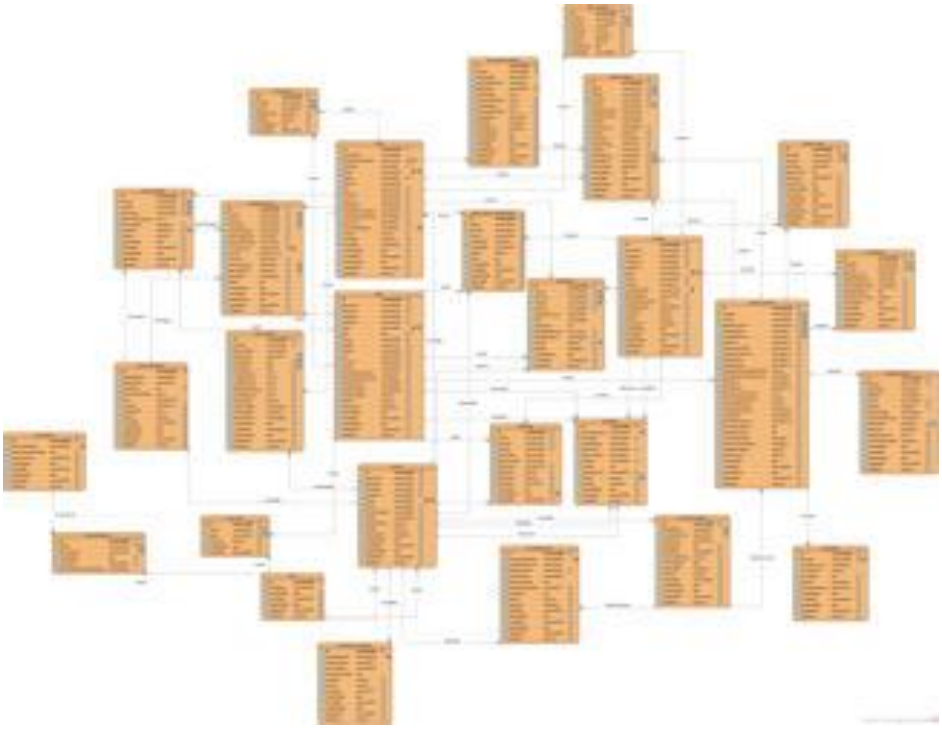
Screen / Role	User	Doctor	Clinic	Admin
Sign In / Sign Up	x	x	x	x
Dashboard	x	x	x	x
Upload Fundus/OCT Images	x		x	
Results & Risk Level	x	x	x	x
Heatmap / Annotations	x	x	x	
Consultation Chat	x	x		
Assigned Cases		x	x	
Bulk Upload			x	
High-Risk Alerts		x	x	x
Checkout / Payment	x		x	x
AI Configuration				x
Account & RBAC Management				x

3.1.4 Non-Screen Functions

No	Function Name	Type	Description
1	Image Analysis Processing	Background Job	Processes Fundus/OCT images using AI models.
2	Risk Scoring Engine	AI Service	Calculates disease risk levels based on analysis results.

3	Heatmap Generation	AI Service	Generates visual heatmaps highlighting abnormal regions.
4	Notification Dispatch	Background Service	Sends alerts and notifications via external gateways.
5	Bulk Job Scheduler	Cron Job	Manages scheduling and execution of bulk analysis jobs.
6	AI Model Feedback Loop	Background Process	Collects doctor feedback to improve AI performance.
7	Payment Processing	External API	Handles secure payment transactions.
8	Report Generation	Backend Service	Generates downloadable reports in PDF or CSV format.
9	Audit Logging	System Service	Records system activities for auditing and compliance.
10	AI Configuration Deployment	Admin Action	Applies updated AI configurations to the core AI service.

3.1.5 Entity Relationship Diagram



4 Non-Functional Requirements

4.1 External Interfaces

4.1.1 Customer/User Interface

The interface must be user-friendly, intuitive, and easy to use for all user groups: User (Patient), Doctor, Clinic, Admin.

The Web UI must be responsive on desktop/laptop/tablet (at minimum), and compatible with common browsers (latest stable Chrome/Edge/Firefox).

The design must prioritize the clinical workflow: upload → job tracking → results → explainability assets → export report → history.

The UI must clearly communicate "CDS only": AI outputs support decision-making and do not
replace the final clinical diagnosis.

4.1.2 Software Interfaces

OAuth/SSO (Google/Social):

+ OAuth2/OIDC; the system receives token/claims and links them to an AURA account.

AI Core Service (Python Inference):

+ Receives images (or image URLs) + job metadata + threshold configuration.

+ Returns scores/risk level/findings + references to explainability assets (heatmap/annotation)

model/version information.

Object Storage (S3/Cloudinary or equivalent):

+ Stores original images, annotated/heatmap assets, and report files (PDF/CSV).

+ Provides access URLs (prefer signed URLs for sensitive data, when applicable).

Notification Gateway (Email/SMS/Push):

+ Sends event-based notifications: job completed, high-risk alerts, billing.

+ Receives delivery status if supported by the provider.

Payment Provider (VNPay/Stripe or sandbox):

+ Creates payment requests for packages/credits.

+ Receives confirmation webhooks to activate packages/add credits and record transactions.

Database (PostgreSQL):

+ Stores metadata, analysis results, history, audit logs, and traceability fields.

Other integrations (optional/recommended for later phases):

+ FHIR/HL7 integration for HIS/EMR interoperability (if expanded to real deployments).

4.1.3 Hardware/Imaging Interfaces

AURA prioritizes “standard file upload” integration rather than vendor-proprietary protocols:

+ Fundus/OCT exported in common formats (JPEG/PNG; DICOM if available).

+ Users/Clinics upload files from imaging devices or internal storage.

The system must validate inputs (size/format/minimum quality) before sending them to the analysis queue.

4.1.4 Communication Interfaces

Web UI ↔ Backend API: HTTPS REST (TLS 1.2+).

Backend API ↔ Orchestrator/Worker/AI Core: internal HTTPS REST or queue-based messaging (deployment-dependent).

Payment Provider → Backend API: HTTPS webhook; signature/secret verification is required.

4.2 Quality Attributes

4.2.1 Usability

The interface must be user-friendly, intuitive, and consistent across roles (User/Doctor/Clinic/Admin), using appropriate clinical terminology.

Critical states must be explicit: Queued/Running/Completed/Failed.

The UI should provide lightweight guidance (tooltips/help) for: image upload, interpreting risk level, viewing heatmap/annotation, exporting reports.

Training & Familiarization Time

User/Patient: ≤ 15 minutes to complete core tasks (sign in, upload, view results, download report).

Doctor: ≤ 60 minutes to master review/validate flow, notes, and trend tracking.

Clinic staff: ~ 60–120 minutes to master bulk upload, job tracking, and campaign exports.

Admin: ~ 120 minutes to manage accounts/RBAC, configure AI threshold/policy, and review audit logs.

Usability Metrics

Establish measurable indicators such as the time required to complete critical tasks. Example (AURA UI): from uploading images to successfully submitting an analysis job should not exceed 2 minutes
under normal network conditions.

Collect user feedback via post-use surveys (patients/clinicians) to assess usability and continuously refine the UI.

4.2.2 Reliability

Availability

The system should maintain a minimum uptime of 99.9% annually during intended service periods.

Schedule regular maintenance during low-traffic periods to minimize disruption.

MTBF (Mean Time Between Failures) and MTTR (Mean Time To Repair)

Target MTBF of over 10,000 continuous operating hours for critical components (API, DB, queue/worker, AI service).

Ensure MTTR for serious incidents does not exceed 30 minutes to minimize operational impact.

Accuracy and Error Control

Ensure that key records (analysis results, audit logs, transactions/credits) are persisted accurately and consistently.

Clearly define error severity levels (e.g., minor, significant, critical), where “critical” errors are those that can
disable core workflows (e.g., job creation, result retrieval, report export).

4.2.3 Performance

Response Time

For interactive transactions (e.g., sign-in, creating a job, opening a result page), average response time should be
under 2 seconds, with a maximum of 5 seconds under high load.

Note: AI inference is an asynchronous workflow; inference latency is handled via job status tracking (see NFR-1/NFR-2).

Throughput

The system must be capable of handling at least 1,000 API transactions per minute during peak usage (subject to infrastructure).

Scalability and Concurrent Users

The system must support horizontal scaling of stateless services (Web/API) and scalable worker pools for queued analysis.

Concurrency should be supported for Clinic bulk workloads without blocking other roles from core actions (viewing results, exports).

5 Requirement Appendix

5.1 Business Rules

ID	Rule Definition
BR-01	accounts email must be unique.
BR-02	Only active/approved accounts may access protected features.
BR-03	One account may have at most one profile type (patient OR doctor).
BR-04	Only approved clinics may use clinic-level features (bulk upload, campaigns, dashboards).
BR-05	Clinic users can access data only within their own clinic id
BR-06	Doctors can access patient cases only when an active assignment exists

BR-07	Every imaging study must be linked to a valid patient id
BR-08	A study must contain ≥ 1 image before analysis can be created
BR-09	Each image must have a stored SHA-256; `retinal_images.sha256` is unique
BR-10	Validate image format/size/minimum quality before enqueuing analysis.
BR-11	Analysis job creation must be blocked when credits are insufficient (unless sandbox/free policy applies).
BR-12	One analysis job produces at most one result
BR-13	Job status transitions are controlled; retry is allowed only by policy
BR-14	Each result must reference ai model version and threshold config.
BR-15	Reports can be generated only after a result exists and must include traceability fields.
BR-16	Only Admin can activate threshold/policy configs; changes must be versioned and auditable.
BR-17	For a model version, only one threshold config should be effective at a time (no overlap)
BR-18	Research/retraining uses consent-controlled data and must be anonymized when used.
BR-19	Notifications can be sent only using active templates; delivery status must be tracked.
BR-20	Grant credits/subscriptions only after verified payment confirmation (signed webhook).
BR-21	Campaigns belong to one clinic; included studies must match the same clinic; statuses are controlled.
BR-22	Only conversation participants may send messages in that conversation.
BR-23	Sensitive actions must generate audit log entries.

5.2 Common Requirements

Authentication and Authorization:

+ AURA must enforce authentication for all protected features.

+ Authorization must be role-based (RBAC) and enforced consistently across UI and APIs.

+ Users can access only the functions and data relevant to their role (User/Doctor/Clinic/Admin).

Data Privacy:

+ Sensitive patient data must be protected in transit and protected at rest where supported.

+ Access to patient-identifiable data must follow least-privilege and be auditable.

+ Data used for research/statistics export or retraining must follow consent policy and must be anonymized.

Data Integrity and Consistency:

+ Core records (uploads, jobs, results, reports, credits/transactions) must be stored reliably and consistently.

+ Traceability must be preserved for every result/report (model version + threshold configuration).

Error Handling and User Feedback:

+ The system must return clear, user-friendly messages for failures (upload, analysis, export, payment).

+ Failures in asynchronous processing must be visible via job status

(Queued/Running/Succeeded/Failed) with an error reason.

Auditability:

+ Security-sensitive actions (login, patient data access, exports, RBAC changes, AI policy changes, billing actions) must generate audit logs.

Scalability:

+ The system must support queued/asynchronous processing for analysis jobs and Clinic bulk workflows.

+ Stateless services (Web/API) and worker pools should support horizontal scaling.

5.3 Application Messages List

Message Code	Message Type	Context	Content
MSG001	Error	Authentication	Invalid credentials. Please try again.
MSG002	Success	Authentication	Signed in successfully
MSG003	Error	Authorization	Access denied: you do not have permission to perform this action.
MSG004	Info	Session	Your session has expired. Please sign in again.
MSG005	Error	OAuth/SSO	SSO sign-in failed. Please try again or use password login.
MSG006	Success	OAuth/SSO	SSO account linked successfully.
MSG007	Error	Upload	Upload failed. Please check your file and network connection.
MSG008	Error	Upload	Unsupported file format or invalid image.
MSG009	Error	Upload	File size exceeds the allowed limit.
MSG010	Success	Upload	Upload completed successfully.
MSG011	Error	Credits/Billing	Insufficient credits to create an analysis job.
MSG012	Success	Analysis Job	Analysis job created and queued
MSG013	Info	Analysis Job	Job is running. Please wait.
MSG014	Success	Analysis Result	Analysis completed. Results are now available.
MSG015	Error	Analysis Result	Analysis failed. Please retry or contact support.
MSG016	Info	Explainability	Heatmap/annotations are available for this result.
MSG017	Error	Report Export	Report export failed. Please try again later.
MSG018	Success	Report Export	Report generated successfully. Your download is ready.
MSG019	Error	Payment	Payment initiation failed. Please try again.
MSG020	Info	Payment	Payment is processing. Please wait for confirmation.

MSG021	Success	Payment	Payment confirmed. Credits/subscription activated.
MSG022	Error	Payment	Payment failed or was cancelled. No credits were added.
MSG023	Error	Consent/Privacy	Consent is required for this operation. Please review and confirm.
MSG024	Success	Consent/Privacy	Consent updated successfully
MSG025	Info	Notification	Notification sent successfully.
MSG026	Error	Notification	Notification delivery failed. Please retry.
MSG027	Success	Messaging/Chat	Message sent.
MSG028	Error	Messaging/Chat	Message failed to send. Please try again.
MSG029	Info	Clinic Campaign	Campaign status updated successfully.
MSG030	Error	Clinic Campaign	Campaign update failed. Please check the campaign status and try again.
MSG031	Error	System	An unexpected error occurred. Please try again later.

IV. Software Design Description

1 System Design

The AURA system (SP26SE025), designed as a Clinical Decision Support (CDS) tool, employs a Microservices Architecture to achieve high scalability, especially for the resource-intensive AI analysis tasks [NFR-7, NFR-17].

1.1 System Architecture

The architecture is composed of decoupled microservices managed via an API Gateway, ensuring security and efficient traffic routing.

1.1.1 Frontend:

The client-side technology stack emphasizes performance and usability [NFR-13]:
+ Technology: React + TypeScript is utilized for building the Web Client, ensuring strong data type checking and extensibility. Flutter is noted as an optional technology for mobile development.

+ Deployment: The Web Application component is deployed in a cloud-based frontend environment (e.g., Vercel or Google Cloud), ensuring high performance and responsiveness [NFR-3].

1.1.2 Backend:

The backend operates on a Microservices principle, allowing independent deployment and scaling.
+ Core Technologies: Services are primarily developed using .NET. The intensive analytical tasks are handled by the dedicated AI Core Microservice, built using Python.
+ Data Management:
Primary Database: PostgreSQL is used for structured data, including user profiles, reports, transaction history, and analysis metadata.
Secondary Database: MongoDB can be utilized for non-relational data, such as real-time messaging data (if applicable) and centralized Audit Logs [17, FR-37].
Storage: Cloud-based storage (e.g., Cloudinary or S3-like services) is mandatory for storing retinal images securely [FR-2].
Communication:
+ Synchronous: All external and most internal service-to-service communication is via RESTful API endpoints [NFR-21].
+ Asynchronous: A Message Queue (MQ) is essential for managing heavy, time-consuming tasks like AI analysis [NFR-1] and Bulk Upload processing [NFR-2].

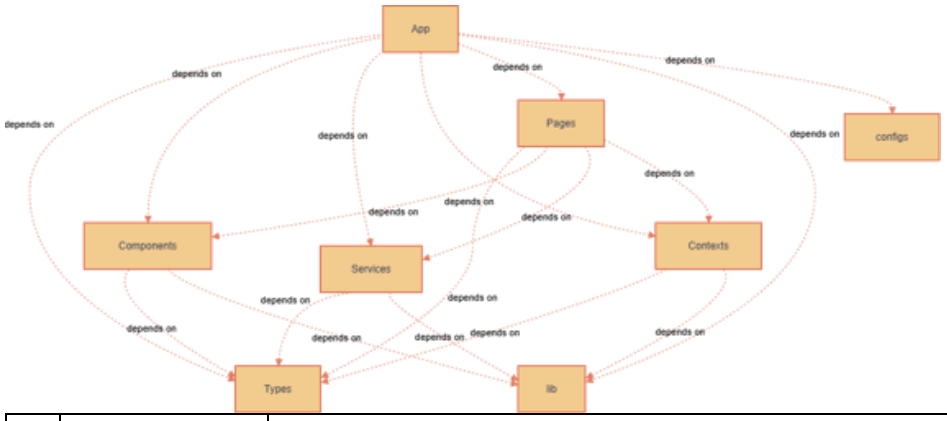
1.1.3 Third-party & Managed Services:

The system leverages managed cloud services for reliability and security:
+ Authentication: Services like Supabase Auth or equivalent are used for token-based authentication (JWT/OAuth2) [16, FR-1].
+ Containerization: Docker is the standard tool for packaging all microservices.
+ Orchestration: Kubernetes (or a managed container service) is implicitly required to manage deployment and execute the horizontal scaling of AI microservices [NFR-7].

IV. Soware Design Description

1.2 Package Diagram

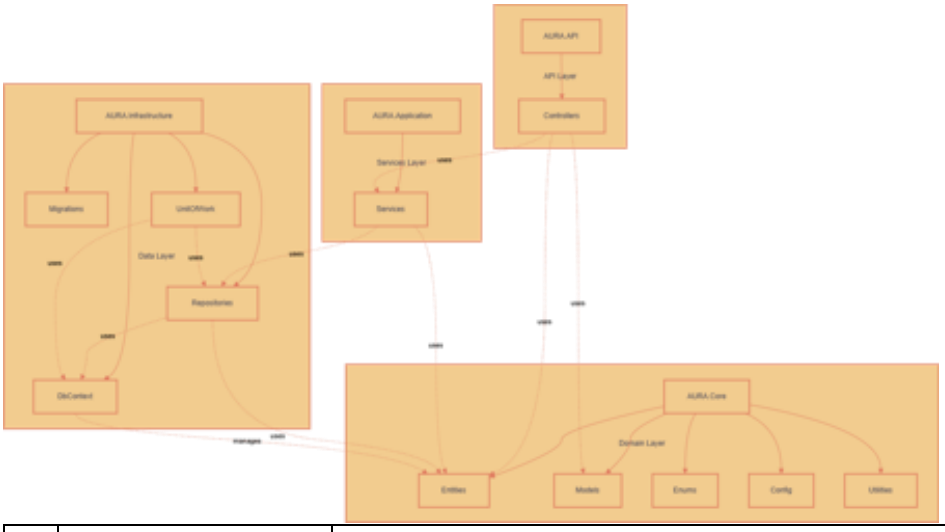
1.2.1 Frontend



no	package	Description
1	src	The root directory contains the entire source code of the application. This is where the modules, UI components, processing logic, and project configuration are located.
2	public	Contains static resources such as images, icons (favicons), or files that need to be served directly from the server without going through the build process (e.g., HTML, JSON).
3	src/app	Contains the core components or main structure of the application. This is where main modules such as pages, features, or important contexts are managed.
4	src/contexts	Contains React Contexts for managing global application state. Includes AuthContext for managing authentication and user state.
5	src/component	Contains common UI components such as buttons, modals, or input forms. Often combined with inline CSS or TailwindCSS for consistent styling.
6	src/pages	Contains page components representing different routes in the application such as Login, Register, Dashboard, AnalysisUpload, AnalysisHistory, AnalysisDetail.
7	src/service	Contains service layer for communicating with backend API. Includes authService and analysisService for handling HTTP requests.
8	src/types	Contains TypeScript type definitions and interfaces used throughout the application such as User, Analysis, Recommendation.
9	src/hooks	Includes custom React Hooks for reusing complex logic, such as API management, dynamic state, or synchronous actions. (Not yet implemented)
10	src/config	Contains the main configuration of the application, including environment variables definition file, API endpoint configuration or theme configuration with TailwindCSS. (Not yet implemented)

11	src/lib	Contains libraries or generic logic source code, for example utility functions or logic processing classes that are not dependent on any specific component. (Not yet implemented)
12	src/layouts	Defines the general layout for pages (such as pages with navigation bars, sidebars, or footers). Helps ensure UI consistency across the application. (Layout component is placed in components)

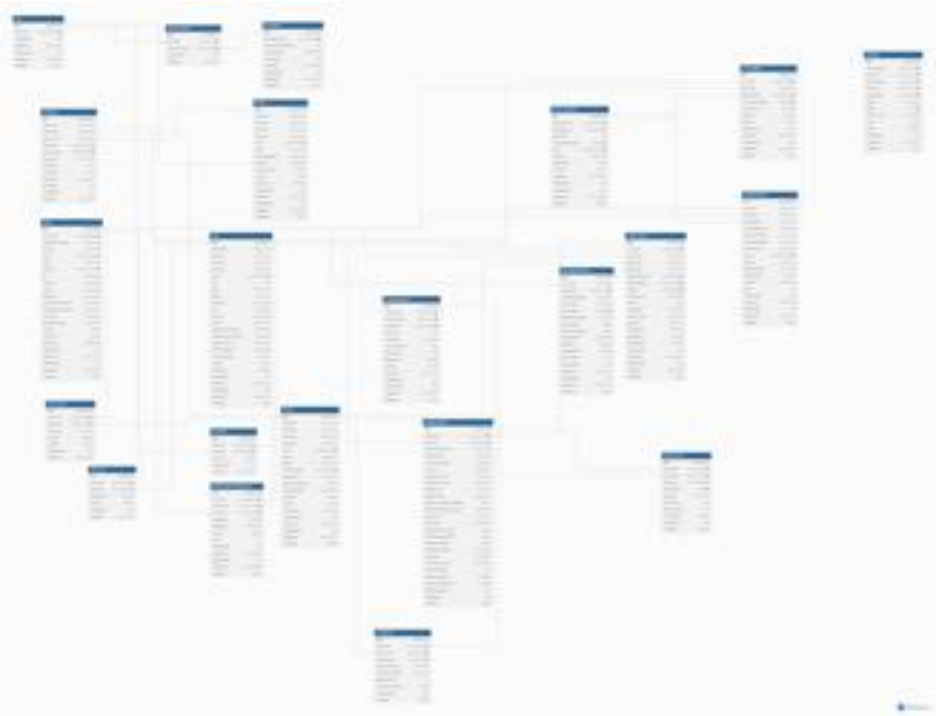
1.2.2 backend.



no	Directory/Package	Description
1	src	The root directory contains the entire source code of the backend application. This is where the modules, controllers, services, and project configuration are located.
2	src/configuration	Holds configuration files required for application setup, such as environment variables, database configurations, and application-level settings.
3	src/controllers	Manages HTTP requests and responses. Controllers expose endpoints for client applications, process incoming requests, and invoke relevant services. Implements authentication using JWT (JSON Web Token) to ensure secure access to protected routes.
4	src/security	Handles security components such as JWT token creation, validation, and user authorization. Provides methods for secure password management and role-based access control (RBAC).
5	src/services	Contains the business logic of the application. Implements methods for processing and managing data across different modules. Key features include JWT Authentication logic integration and a method to send requests to AI Core service for AI-powered responses.

6	src/utils	Holds utility classes and helper functions for general-purpose tasks. These tasks include string manipulation, date formatting, and file management.
7	src/mappers	Provides mapping logic to convert data between different layers (e.g., entity-to-DTO conversion). Its role is to ensure a clean and structured data flow between various components of the application.
8	src/exceptions	Responsible for managing custom exceptions and error handling for the application. Ensures clear and consistent error responses for both HTTP and service-level operations.
9	src/repositories	Acts as the Data Access Layer (DAL). Provides Create, Read, Update, and Delete (CRUD) operations for interacting with databases.
10	src/specification	Defines query specifications and search filters. Its purpose is to simplify database queries and enhance the maintainability of the code.
11	src/models	Houses domain models and data structures that represent entities and objects used across the entire application.
12	src/resources	Contains static files or resource files required by the application, such as configurations, templates, or shared assets.

2. **Database Design**



2.1 Table: roles

Field Name	Type	Description	Unique	Not null	PK,FK
Id	varchar(255)	Unique identifier for the role	yes	yes	PK
RoleName	varchar(255)	Name of the role	no	yes	no
CreatedDate	date	Date when the record was created	no	no	no
CreatedBy	varchar(255)	User who created the record	no	no	no
UpdatedDate	date	Date when the record was last updated	no	no	no
UpdatedBy	varchar(255)	User who last updated the record	no	no	no
IsDeleted	boolean	Soft delete flag (FALSE=active, TRUE=deleted)	no	no	no

2.2 Table: permissions

Field Name	Type	Description	Unique	Not null	PK,FK
Id	varchar(255)	Unique identifier for the permission	yes	yes	PK
PermissionName	varchar(255)	Unique code/name of the permission	yes	yes	no
PermissionDescription	text	Detailed description of the permission	no	no	no
ResourceType	varchar(255)	Type of resource the permission applies to	no	no	no
CreatedDate	date	Date when the record was created	no	no	no
CreatedBy	varchar(255)	User who created the record	no	no	no
UpdatedDate	date	Date when the record was last updated	no	no	no
UpdatedBy	varchar(255)	User who last updated the record	no	no	no
IsDeleted	boolean	Soft delete flag (FALSE=active, TRUE=deleted)	no	no	no

2.3 Table: role_permissions

Field Name	Type	Description	Unique	Not null	PK,FK
Id	varchar(255)	Unique identifier for the mapping	yes	yes	PK
RoleId	varchar(255)	Foreign key reference to Roles table	no	yes	FK
PermissionId	varchar(255)	Foreign key reference to Permissions table	no	yes	FK
CreatedDate	date	Date when the mapping was created	no	no	no
CreatedBy	varchar(255)	User who created the mapping	no	no	no

2.4 Table: users

Field Name	Type	Description	Unique	Not null	PK,FK
Id	varchar(255)	Unique identifier for the user	yes	yes	PK
Username	varchar(255)	User's login username	no	no	no
Password	varchar(255)	Hashed password	no	no	no
FirstName	varchar(255)	User's first name	no	no	no
LastName	varchar(255)	User's last name	no	no	no
Email	varchar(255)	User's email address	yes	yes	no
Dob	date	Date of birth	no	no	no
Phone	varchar(255)	Phone number	no	no	no
Gender	varchar(10)	Gender (Male, Female, Other)	no	no	no
Address	varchar(255)	Street address	no	no	no
City	varchar(100)	City name	no	no	no
Province	varchar(100)	Province or state name	no	no	no
Country	varchar(100)	Country name (Default: Vietnam)	no	no	no
IdentificationNumber	varchar(50)	Citizen ID or Passport number	no	no	no

AuthenticationProvider	varchar(50)	Auth provider (email, google, facebook)	no	no	no
ProviderUserId	varchar(255)	ID from external provider	no	no	no
ProfileImageUrl	varchar(500)	URL to profile picture	no	no	no
IsEmailVerified	boolean	Email verification status	no	no	no
IsActive	boolean	Account active status	no	no	no
LastLoginAt	timestamp	Timestamp of last login	no	no	no
CreatedDate	date	Date created	no	no	no
CreatedBy	varchar(255)	User who created the record	no	no	no
UpdatedDate	date	Date last updated	no	no	no
UpdatedBy	varchar(255)	User who last updated	no	no	no
IsDeleted	boolean	Soft delete flag	no	no	no

2.5 Table: doctors

Field Name	**Type**	**Description**	**Unique**	**Not null**	**PK,FK**
Id	varchar(255)	Unique identifier for the doctor	yes	yes	PK
Username	varchar(255)	Doctor's login username	no	no	no
Password	varchar(255)	Hashed password	no	no	no
FirstName	varchar(255)	Doctor's first name	no	no	no
LastName	varchar(255)	Doctor's last name	no	no	no
Email	varchar(255)	Doctor's email address	yes	yes	no
Phone	varchar(255)	Phone number	no	no	no
Gender	varchar(10)	Gender (Male, Female, Other)	no	no	no
LicenseNumber	varchar(100)	Medical license number	yes	yes	no
Specialization	varchar(255)	Medical specialization area	no	no	no
YearsOfExperience	integer	Number of years of experience	no	no	no
ProfileImageUrl	varchar(500)	URL to profile picture	no	no	no
IsVerified	boolean	Verification status of license	no	no	no

IsActive	boolean	Account active status	no	no	no
LastLoginAt	timestamp	Timestamp of last login	no	no	no
CreatedDate	date	Date created	no	no	no
CreatedBy	varchar(255)	User who created the record	no	no	no
UpdatedDate	date	Date last updated	no	no	no
UpdatedBy	varchar(255)	User who last updated	no	no	no
IsDeleted	boolean	Soft delete flag	no	no	no

2.6 Table: clinics

Field Name	**Type**	**Description**	**Unique**	**Not null**	**PK,FK**
Id	varchar(255)	Unique identifier for the clinic	yes	yes	PK
ClinicName	varchar(255)	Name of the clinic	no	yes	no
RegistrationNumber	varchar(100)	Business/Medical registration number	yes	no	no
TaxCode	varchar(50)	Tax identification number	no	no	no
Email	varchar(255)	Contact email address	yes	yes	no
Phone	varchar(255)	Contact phone number	no	no	no
Address	varchar(255)	Physical address	no	yes	no
City	varchar(100)	City name	no	no	no
Province	varchar(100)	Province or state name	no	no	no
Country	varchar(100)	Country name (Default: Vietnam)	no	no	no
WebsiteUrl	varchar(500)	Website URL	no	no	no
ContactPersonName	varchar(255)	Name of primary contact person	no	no	no
ContactPersonPhone	varchar(255)	Phone of primary contact person	no	no	no
ClinicType	varchar(50)	Type (Hospital, Clinic, etc.)	no	no	no
VerificationStatus	varchar(50)	Status (Pending, Approved, etc.)	no	no	no

IsActive	boolean	Operational status	no	no	no
VerifiedAt	timestamp	Timestamp when verified	no	no	no
VerifiedBy	varchar(255)	Admin who verified the clinic	no	no	FK
CreatedDate	date	Date created	no	no	no
CreatedBy	varchar(255)	User who created the record	no	no	no
UpdatedDate	date	Date last updated	no	no	no
UpdatedBy	varchar(255)	User who last updated	no	no	no
IsDeleted	boolean	Soft delete flag	no	no	no

2.7 Table: admins

Field Name	**Type**	**Description**	**Unique**	**Not null**	**PK,FK**
Id	varchar(255)	Unique identifier for the admin	yes	yes	PK
Username	varchar(255)	Admin's login username	no	no	no
Password	varchar(255)	Hashed password	no	no	no
FirstName	varchar(255)	Admin's first name	no	no	no
LastName	varchar(255)	Admin's last name	no	no	no
Email	varchar(255)	Admin's email address	yes	yes	no
Phone	varchar(255)	Phone number	no	no	no
ProfileImageUrl	varchar(500)	URL to profile picture	no	no	no
RoleId	varchar(255)	Foreign key to Role table	no	yes	FK
IsSuperAdmin	boolean	Super admin privilege flag	no	no	no
IsActive	boolean	Account active status	no	no	no
LastLoginAt	timestamp	Timestamp of last login	no	no	no
CreatedDate	date	Date created	no	no	no
CreatedBy	varchar(255)	User who created the record	no	no	no
UpdatedDate	date	Date last updated	no	no	no
UpdatedBy	varchar(255)	User who last updated	no	no	no
IsDeleted	boolean	Soft delete flag	no	no	no

2.8 Table: user_roles

Field Name	Type	Description	Unique	Not null	PK,FK
Id	varchar(255)	Unique identifier for the relationship	yes	yes	PK
UserId	varchar(255)	Foreign key reference to Users table	no	yes	FK
RoleId	varchar(255)	Foreign key reference to Roles table	no	yes	FK
IsPrimary	boolean	Indicates if this is the primary role	no	no	no
CreatedDate	date	Date when the mapping was created	no	no	no
CreatedBy	varchar(255)	User who created the mapping	no	no	no

2.9 Table: clinic_doctors

Field Name	Type	Description	Unique	Not null	PK,FK
Id	varchar(255)	Unique identifier for the relationship	yes	yes	PK
ClinicId	varchar(255)	Foreign key reference to Clinics table	no	yes	FK
DoctorId	varchar(255)	Foreign key reference to Doctors table	no	yes	FK
IsPrimary	boolean	Indicates if this is the primary clinic	no	no	no
JoinedAt	timestamp	Timestamp when doctor joined the clinic	no	no	no
IsActive	boolean	Active status of the employment	no	no	no
CreatedDate	date	Date created	no	no	no
CreatedBy	varchar(255)	User who created the record	no	no	no

2.10 Table: clinic_users

Field Name	Type	Description	Unique	Not null	PK,FK
Id	varchar(255)	Unique identifier for the relationship	yes	yes	PK
ClinicId	varchar(255)	Foreign key reference to Clinics table	no	yes	FK
UserId	varchar(255)	Foreign key reference to Users table	no	yes	FK
RegisteredAt	timestamp	Timestamp when user registered at clinic	no	no	no
IsActive	boolean	Active status of the registration	no	no	no

CreatedDate	date	Date created	no	no	no
CreatedBy	varchar(255)	User who created the record	no	no	no

2.11 Table: patient_doctor_assignments

Field Name	**Type**	**Description**	**Unique**	**Not null**	**PK,FK**
Id	varchar(255)	Unique identifier for the assignment	yes	yes	PK
UserId	varchar(255)	Foreign key reference to Users (Patient)	no	yes	FK
DoctorId	varchar(255)	Foreign key reference to Doctors	no	yes	FK
ClinicId	varchar(255)	Foreign key reference to Clinics	no	no	FK
AssignedAt	timestamp	Timestamp when assignment was made	no	no	no
AssignedBy	varchar(255)	Admin who made the assignment	no	no	FK
IsActive	boolean	Active status of the assignment	no	no	no
Notes	text	Additional notes regarding assignment	no	no	no
CreatedDate	date	Date created	no	no	no
CreatedBy	varchar(255)	User who created the record	no	no	no
UpdatedDate	date	Date last updated	no	no	no
UpdatedBy	varchar(255)	User who last updated	no	no	no
IsDeleted	boolean	Soft delete flag	no	no	no

2.12 Table: ai_model_versions

Field Name	**Type**	**Description**	**Unique**	**Not null**	**PK,FK**
Id	varchar(255)	Unique identifier for the model version	yes	yes	PK
ModelName	varchar(255)	Name of the AI model	no	yes	no
VersionNumber	varchar(50)	Version identifier (e.g., v1.0, v2.1)	no	yes	no
ModelType	varchar(100)	Type of architecture/task	no	yes	no

Description	text	Detailed description of this version	no	no	no
ModelPath	varchar(500)	File path or URL to the model artifact	no	no	no
AccuracyMetrics	jsonb	JSON object containing performance metrics	no	no	no
TrainingDate	date	Date when the model was trained	no	no	no
DeployedAt	timestamp	Timestamp when the model was deployed	no	no	no
IsActive	boolean	Operational status of this version	no	no	no
CreatedBy	varchar(255)	Admin who created the version	no	no	FK
CreatedDate	date	Date created	no	no	no
UpdatedDate	date	Date last updated	no	no	no
UpdatedBy	varchar(255)	User who last updated	no	no	no
IsDeleted	boolean	Soft delete flag	no	no	no

2.12 Table: retinal_images

Field Name	**Type**	**Description**	**Unique**	**Not null**	**PK,FK**
Id	varchar(255)	Unique identifier for the image	yes	yes	PK
UserId	varchar(255)	Owner of the image (Patient)	no	yes	FK
ClinicId	varchar(255)	Clinic associated with the image	no	no	FK
DoctorId	varchar(255)	Doctor associated with the image	no	no	FK
BatchId	varchar(255)	Bulk upload batch reference	no	no	FK
OriginalFilename	varchar(255)	Original name of the uploaded file	no	yes	no
StoredFilename	varchar(255)	System generated filename for storage	no	yes	no
FilePath	varchar(500)	Path to file in storage system	no	yes	no
CloudinaryUrl	varchar(500)	CDN URL for quick access	no	no	no
FileSize	bigint	Size of the file in bytes	no	no	no
ImageType	varchar(50)	Type of scan (Fundus, OCT)	no	no	no

ImageFormat	varchar(10)	File format (JPEG, PNG, DICOM...)	no	no	no
EyeSide	varchar(10)	Left or Right eye	no	no	no
UploadStatus	varchar(50)	Status (Uploaded, Processing, Failed)	no	no	no
UploadedAt	timestamp	Timestamp when upload occurred	no	no	no
ProcessedAt	timestamp	Timestamp when processing finished	no	no	no
CreatedDate	date	Date created	no	no	no
CreatedBy	varchar(255)	User who created the record	no	no	no
UpdatedDate	date	Date last updated	no	no	no
UpdatedBy	varchar(255)	User who last updated	no	no	no
IsDeleted	boolean	Soft delete flag	no	no	no

2.13 Table: bulk_upload_batches

Field Name	**Type**	**Description**	**Unique**	**Not null**	**PK,FK**
Id	varchar(255)	Unique identifier for the batch	yes	yes	PK
ClinicId	varchar(255)	Clinic performing the upload	no	yes	FK
UploadedBy	varchar(255)	User ID performing the upload	no	yes	no
UploadedByType	varchar(20)	User role (Doctor, Admin, Manager)	no	yes	no
BatchName	varchar(255)	Friendly name for the batch	no	no	no
TotalImages	integer	Total number of images in batch	no	yes	no
ProcessedImages	integer	Count of successfully processed images	no	no	no
FailedImages	integer	Count of failed images	no	no	no
ProcessingImages	integer	Count of currently processing images	no	no	no
UploadStatus	varchar(50)	Batch status (Pending, Completed...)	no	no	no
StartedAt	timestamp	Timestamp when batch started	no	no	no

CompletedAt	timestamp	Timestamp when batch finished	no	no	no
FailureReason	text	Reason for failure if applicable	no	no	no
CreatedDate	date	Date created	no	no	no
CreatedBy	varchar(255)	User who created the record	no	no	no
UpdatedDate	date	Date last updated	no	no	no
UpdatedBy	varchar(255)	User who last updated	no	no	no
IsDeleted	boolean	Soft delete flag	no	no	no

2.14 Table: analysis_results

Field Name	**Type**	**Description**	**Unique**	**Not null**	**PK,FK**
Id	varchar(255)	Unique identifier for the result	yes	yes	PK
ImageId	varchar(255)	Reference to the analyzed image	no	yes	FK
UserId	varchar(255)	Reference to the patient	no	yes	FK
ModelVersionId	varchar(255)	AI Model version used	no	yes	FK
AnalysisStatus	varchar(50)	Status (Processing, Completed...)	no	no	no
OverallRiskLevel	varchar(50)	Overall assessment (Low, High...)	no	no	no
RiskScore	decimal(5,2)	Aggregate risk score (0-100)	no	no	no
HypertensionRisk	varchar(50)	Risk level for hypertension	no	no	no
HypertensionScore	decimal(5,2)	Score for hypertension risk	no	no	no
DiabetesRisk	varchar(50)	Risk level for diabetes	no	no	no
DiabetesScore	decimal(5,2)	Score for diabetes risk	no	no	no
DiabeticRetinopathyDetected	boolean	Detection flag for DR	no	no	no
DiabeticRetinopathySeverity	varchar(50)	Severity scale (Mild, Severe...)	no	no	no

StrokeRisk	varchar(50)	Risk level for stroke	no	no	no
StrokeScore	decimal(5,2)	Score for stroke risk	no	no	no
MicroaneurysmsCount	integer	Count of microaneurysms detected	no	no	no
HemorrhagesDetected	boolean	Flag for hemorrhages	no	no	no
ExudatesDetected	boolean	Flag for exudates	no	no	no
AnnotatedImageUrl	varchar(500)	URL to image with AI markings	no	no	no
HeatmapUrl	varchar(500)	URL to AI attention heatmap	no	no	no
AiConfidenceScore	decimal(5,2)	AI confidence level (0-100)	no	no	no
Recommendations	text	Suggested next steps	no	no	no
AnalysisStartedAt	timestamp	Start time of analysis	no	no	no
AnalysisCompletedAt	timestamp	End time of analysis	no	no	no
DetailedFindings	jsonb	Full JSON structure of findings	no	no	no
CreatedDate	date	Date created	no	no	no
IsDeleted	boolean	Soft delete flag	no	no	no

2.15 Table: annotations

Field Name	**Type**	**Description**	**Unique**	**Not null**	**PK,FK**
Id	varchar(255)	Unique identifier for the result	yes	yes	PK
ImageId	varchar(255)	Reference to the analyzed image	no	yes	FK
UserId	varchar(255)	Reference to the patient	no	yes	FK
ModelVersionId	varchar(255)	AI Model version used	no	yes	FK
AnalysisStatus	varchar(50)	Status (Processing, Completed...)	no	no	no
OverallRiskLevel	varchar(50)	Overall assessment (Low, High...)	no	no	no

RiskScore	decimal(5,2)	Aggregate risk score (0-100)	no	no	no
HypertensionRisk	varchar(50)	Risk level for hypertension	no	no	no
HypertensionScore	decimal(5,2)	Score for hypertension risk	no	no	no
DiabetesRisk	varchar(50)	Risk level for diabetes	no	no	no
DiabetesScore	decimal(5,2)	Score for diabetes risk	no	no	no

2.16 Table: ai_feedback

Field Name	**Type**	**Description**	**Unique**	**Not null**	**PK,FK**
Id	varchar(255)	Unique identifier for the feedback	yes	yes	PK
ResultId	varchar(255)	Reference to the analysis result	no	yes	FK
DoctorId	varchar(255)	Doctor providing the feedback	no	yes	FK
FeedbackType	varchar(50)	Type (Correct, Incorrect, etc.)	no	yes	no
OriginalRiskLevel	varchar(50)	Risk level AI predicted	no	no	no
CorrectedRiskLevel	varchar(50)	Risk level doctor corrected	no	no	no
FeedbackNotes	text	Detailed comments from doctor	no	no	no
IsUsedForTraining	boolean	Flag if used for model retraining	no	no	no
CreatedDate	date	Date created	no	no	no
IsDeleted	boolean	Soft delete flag	no	no	no
Field Name	**Type**	**Description**	**Unique**	**Not null**	**PK,FK**

2.17 Table: medical_notes

Field Name	**Type**	**Description**	**Unique**	**Not null**	**PK,FK**
Id	varchar(255)	Unique identifier for the note	yes	yes	PK
ResultId	varchar(255)	Reference to the analysis result	no	yes	FK
DoctorId	varchar(255)	Doctor creating the note	no	yes	FK
NoteType	varchar(50)	Type (Diagnosis, Prescription, etc.)	no	yes	no

NoteContent	text	Main content of the note	no	yes	no
Diagnosis	varchar(255)	Formal diagnosis code/text	no	no	no
Prescription	text	Medication details	no	no	no
FollowUpDate	date	Recommended return date	no	no	no
IsImportant	boolean	Flag for important notes	no	no	no
CreatedDate	date	Date created	no	no	no
IsDeleted	boolean	Soft delete flag	no	no	no

2.18 Table: service_packages

Field Name	**Type**	**Description**	**Unique**	**Not null**	**PK,FK**
Id	varchar(255)	Unique identifier for the service package	yes	yes	PK
PackageName	varchar(255)	Name of the package	no	yes	no
PackageType	varchar(50)	Type (Individual, Clinic, Enterprise)	no	yes	no
Description	text	Detailed description of benefits	no	no	no
NumberOfAnalyses	integer	Number of AI analyses included	no	yes	no
Price	decimal(10,2)	Cost of the package	no	yes	no
Currency	varchar(10)	Currency code (Default: VND)	no	no	no
ValidityDays	integer	Duration of package validity in days	no	no	no
IsActive	boolean	Availability status for purchase	no	no	no
CreatedBy	varchar(255)	Admin who created the package	no	no	FK
CreatedDate	date	Date created	no	no	no
UpdatedDate	date	Date last updated	no	no	no
UpdatedBy	varchar(255)	User who last updated	no	no	no
IsDeleted	boolean	Soft delete flag	no	no	no

2.19 Table: user_packages

Field Name	Type	Description	Unique	Not null	PK,FK
Id	varchar(255)	Unique identifier for the purchased package	yes	yes	PK
UserId	varchar(255)	Owner ID (if individual user)	no	no	FK
ClinicId	varchar(255)	Owner ID (if clinic)	no	no	FK
PackageId	varchar(255)	Reference to the Service Package	no	yes	FK
RemainingAnalyses	integer	Count of remaining AI uses	no	yes	no
PurchasedAt	timestamp	Timestamp of purchase	no	no	no
ExpiresAt	timestamp	Expiration timestamp	no	no	no
IsActive	boolean	Current active status	no	no	no
CreatedDate	date	Date created	no	no	no
CreatedBy	varchar(255)	User who created the record	no	no	no
UpdatedDate	date	Date last updated	no	no	no
UpdatedBy	varchar(255)	User who last updated	no	no	no
IsDeleted	boolean	Soft delete flag	no	no	no

2.20 Table: payment_history

Field Name	Type	Description	Unique	Not null	PK,FK
Id	varchar(255)	Unique identifier for the transaction	yes	yes	PK
UserId	varchar(255)	Payer ID (if individual)	no	no	FK
ClinicId	varchar(255)	Payer ID (if clinic)	no	no	FK
PackageId	varchar(255)	Reference to the package purchased	no	yes	FK
UserPackageId	varchar(255)	Reference to the resulting user package	no	no	FK
PaymentMethod	varchar(50)	Method (CreditCard, E-Wallet, etc.)	no	no	no

PaymentProvider	varchar(100)	Provider name (e.g., Stripe, Momo)	no	no	no
TransactionId	varchar(255)	External transaction ID from provider	yes	no	no
Amount	decimal(10,2)	Transaction amount	no	yes	no
Currency	varchar(10)	Currency code	no	no	no
PaymentStatus	varchar(50)	Status (Pending, Completed, Failed)	no	no	no
PaymentDate	timestamp	Timestamp of payment	no	no	no
ReceiptUrl	varchar(500)	URL to digital receipt	no	no	no
Notes	text	Additional transaction notes	no	no	no
CreatedDate	date	Date created	no	no	no
CreatedBy	varchar(255)	User who created the record	no	no	no
UpdatedDate	date	Date last updated	no	no	no
UpdatedBy	varchar(255)	User who last updated	no	no	no
IsDeleted	boolean	Soft delete flag	no	no	no

2.21 Table: messages

Field Name	**Type**	**Description**	**Unique**	**Not null**	**PK,FK**
Id	varchar(255)	Unique identifier for the message	yes	yes	PK
ConversationId	varchar(255)	Identifier grouping messages in a thread	no	yes	no
SendById	varchar(255)	Identifier of the sender	no	yes	no
SendByType	varchar(20)	Sender type (User, Doctor, Admin)	no	yes	no
ReceiverId	varchar(255)	Identifier of the receiver	no	yes	no
ReceiverType	varchar(20)	Receiver type (User, Doctor, Admin)	no	yes	no
Subject	varchar(255)	Subject line of the message	no	no	no
Content	text	Main content of the message	no	yes	no
AttachmentUrl	varchar(500)	URL to any attached file	no	no	no

IsRead	boolean	Read status (FALSE=unread, TRUE=read)	no	no	no
ReadAt	timestamp	Timestamp when message was read	no	no	no
CreatedDate	date	Date created	no	no	no
CreatedBy	varchar(255)	User who created the record	no	no	no
IsDeleted	boolean	Soft delete flag	no	no	no

2.22 Table: notifications

Field Name	**Type**	**Description**	**Unique**	**Not null**	**PK,FK**
Id	varchar(255)	Unique identifier for the notification	yes	yes	PK
UserId	varchar(255)	Recipient ID (if User/Patient)	no	no	FK
DoctorId	varchar(255)	Recipient ID (if Doctor)	no	no	FK
ClinicId	varchar(255)	Recipient ID (if Clinic)	no	no	FK
AdminId	varchar(255)	Recipient ID (if Admin)	no	no	FK
NotificationType	varchar(50)	Type (AnalysisComplete, PaymentSuccess...)	no	yes	no
Title	varchar(255)	Title of the notification	no	yes	no
Description	text	Detailed content of the notification	no	yes	no
RelatedEntityType	varchar(50)	Type of linked entity (e.g., AnalysisResult)	no	no	no
RelatedEntityId	varchar(255)	ID of the linked entity	no	no	no
IsRead	boolean	Read status (FALSE=unread, TRUE=read)	no	no	no
ReadAt	timestamp	Timestamp when notification was read	no	no	no
CreatedDate	date	Date created	no	no	no
CreatedBy	varchar(255)	User who created the record	no	no	no
IsDeleted	boolean	Soft delete flag	no	no	no

2.23 Table: exported_reports

Field Name	Type	Description	Unique	Not null	PK,FK
Id	varchar(255)	Unique identifier for the exported report record	yes	yes	PK
ResultId	varchar(255)	Reference to specific analysis result (if applicable)	no	no	FK
ReportType	varchar(50)	Format of report (PDF, CSV, Excel, JSON)	no	yes	no
FilePath	varchar(500)	Internal storage path of the generated file	no	yes	no
FileUrl	varchar(500)	Download URL for the file	no	no	no
FileSize	bigint	Size of the file in bytes	no	no	no
RequestedBy	varchar(255)	User ID who requested the export	no	yes	no
RequestedByType	varchar(20)	Role of requester (User, Doctor, Admin, Clinic)	no	yes	no
ExportedAt	timestamp	Timestamp when report was generated	no	no	no
ExpiresAt	timestamp	Timestamp when the download link expires	no	no	no
DownloadCount	integer	Number of times file has been downloaded	no	no	no
LastDownloadedAt	timestamp	Timestamp of the last download activity	no	no	no
CreatedDate	date	Date created	no	no	no
CreatedBy	varchar(255)	User who created the record	no	no	no
UpdatedDate	date	Date last updated	no	no	no
UpdatedBy	varchar(255)	User who last updated	no	no	no
IsDeleted	boolean	Soft delete flag	no	no	no

2.24 Table: clinic_reports

Field Name	Type	Description	Unique	Not null	PK,FK

Id	varchar(255)	Unique identifier for the clinic report	yes	yes	PK
ClinicId	varchar(255)	Reference to the clinic owning the report	no	yes	FK
ReportName	varchar(255)	Display name of the report	no	yes	no
ReportType	varchar(50)	Type (MonthlySummary, RiskDistribution, etc.)	no	yes	no
PeriodStart	date	Start date of the reporting period	no	no	no
PeriodEnd	date	End date of the reporting period	no	no	no
TotalPatients	integer	Total patients processed in period	no	no	no
TotalAnalyses	integer	Total analyses performed in period	no	no	no
HighRiskCount	integer	Count of high-risk cases identified	no	no	no
MediumRiskCount	integer	Count of medium-risk cases identified	no	no	no
LowRiskCount	integer	Count of low-risk cases identified	no	no	no
ReportData	jsonb	JSON object containing detailed statistics	no	no	no
GeneratedBy	varchar(255)	Admin who generated the report	no	no	FK
GeneratedAt	timestamp	Timestamp when report was generated	no	no	no
ReportFileUrl	varchar(500)	URL to the generated report file (if any)	no	no	no
CreatedDate	date	Date created	no	no	no
CreatedBy	varchar(255)	User who created the record	no	no	no
UpdatedDate	date	Date last updated	no	no	no
UpdatedBy	varchar(255)	User who last updated	no	no	no
IsDeleted	boolean	Soft delete flag	no	no	no

2.25 Table: ai_configurations

Field Name	**Type**	**Description**	**Unique**	**Not null**	**PK,FK**

Id	varchar(255)	Unique identifier for the configuration	yes	yes	PK
ConfigurationName	varchar(255)	Unique name/key of the configuration	yes	yes	no
ConfigurationType	varchar(50)	Type (Threshold, Parameter, Policy)	no	yes	no
ModelVersionId	varchar(255)	Reference to specific AI model version	no	no	FK
ParameterKey	varchar(255)	Key identifier for the parameter	no	yes	no
ParameterValue	text	Value of the parameter	no	yes	no
ParameterDataType	varchar(50)	Data type (Number, String, Boolean, JSON)	no	no	no
Description	text	Explanation of what this config controls	no	no	no
IsActive	boolean	Operational status of the config	no	no	no
AppliedAt	timestamp	Timestamp when config became active	no	no	no
AppliedBy	varchar(255)	Admin who applied the configuration	no	no	FK
CreatedDate	date	Date created	no	no	no
CreatedBy	varchar(255)	User who created the record	no	no	no
UpdatedDate	date	Date last updated	no	no	no
UpdatedBy	varchar(255)	User who last updated	no	no	no
IsDeleted	boolean	Soft delete flag	no	no	no

2.26 Table: notification_templates

Field Name	**Type**	**Description**	**Unique**	**Not null**	**PK,FK**
Id	varchar(255)	Unique identifier for the template	yes	yes	PK
TemplateName	varchar(255)	Unique code name for the template	yes	yes	no
TemplateType	varchar(50)	Context type (AnalysisComplete, SystemAlert...)	no	yes	no
TitleTemplate	varchar(500)	Template for the notification title	no	yes	no

ContentTemplate	text	Template for the notification body	no	yes	no
Variables	jsonb	List of dynamic variables used in template	no	no	no
IsActive	boolean	Status of the template usage	no	no	no
Language	varchar(10)	Language code (e.g., 'vi', 'en')	no	no	no
CreatedDate	date	Date created	no	no	no
CreatedBy	varchar(255)	User who created the record	no	no	no
UpdatedDate	date	Date last updated	no	no	no
UpdatedBy	varchar(255)	User who last updated	no	no	no
IsDeleted	boolean	Soft delete flag	no	no	no

2.27 Table: audit_logs

Field Name	**Type**	**Description**	**Unique**	**Not null**	**PK,FK**
Id	varchar(255)	Unique identifier for the audit log entry	yes	yes	PK
UserId	varchar(255)	Reference to User who performed action (if User)	no	no	FK
DoctorId	varchar(255)	Reference to Doctor who performed action (if Doctor)	no	no	FK
AdminId	varchar(255)	Reference to Admin who performed action (if Admin)	no	no	FK
ActionType	varchar(100)	Type of action (Create, Update, Delete, Login...)	no	yes	no
ResourceType	varchar(100)	Target resource (e.g., User, Image, Report)	no	yes	no
ResourceId	varchar(255)	ID of the specific target resource	no	no	no
OldValues	jsonb	Snapshot of data before change (for updates)	no	no	no
NewValues	jsonb	Snapshot of data after change	no	no	no
IpAddress	varchar(45)	IP address of the client performing action	no	no	no
UserAgent	text	Browser/Device information string	no	no	no

CreatedDate	date	Date when action occurred	no	no	no
CreatedBy	varchar(255)	ID string of the actor (redundant for quick access)	no	no	no

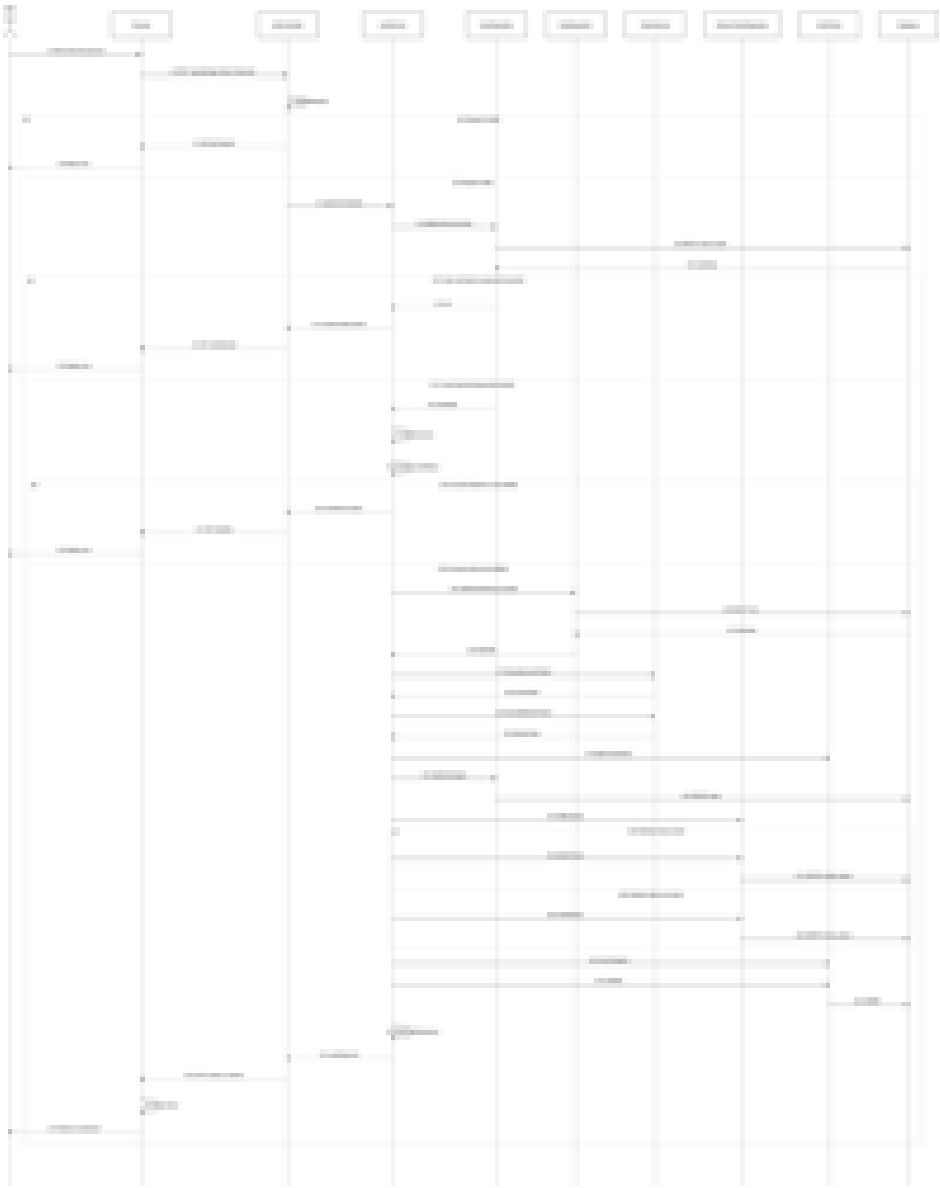
3 Detailed Design

3.1 Authentication

3.1.1 Login

3.1.1.1 Class Diagram

3.1.1.2 Sequence Diagram



3.2 User & Doctor Management

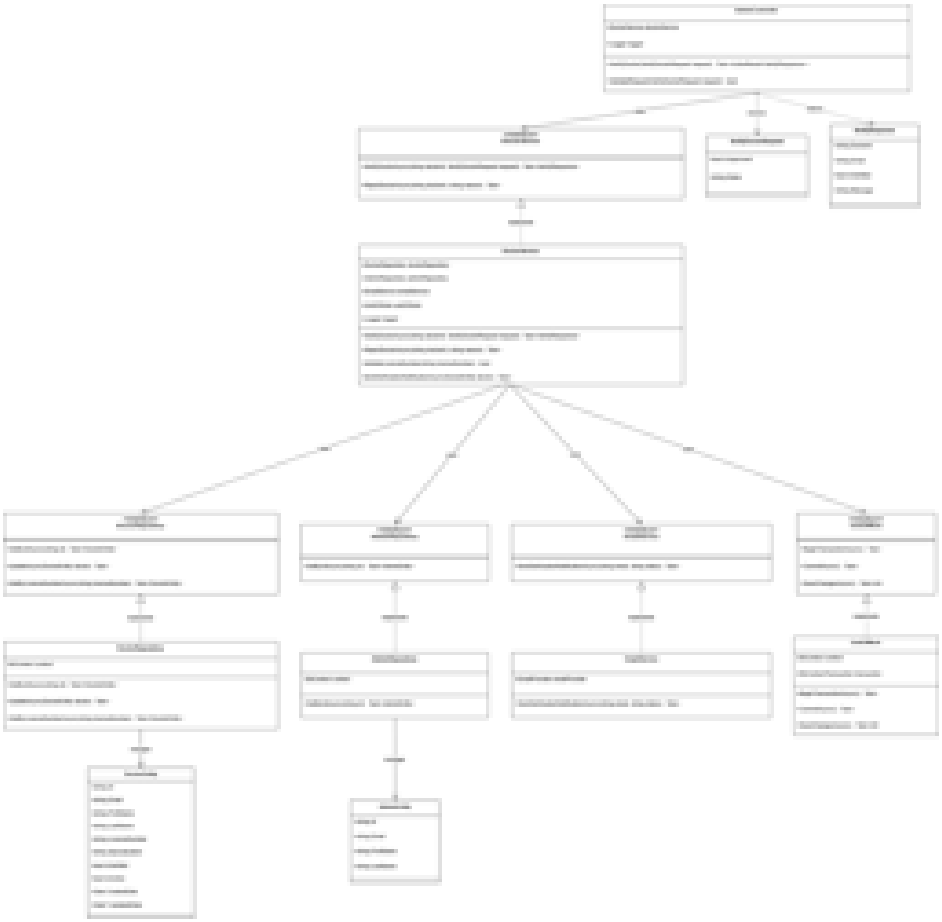
3.2.1 Register Patient

3.2.1.1 Class Diagram

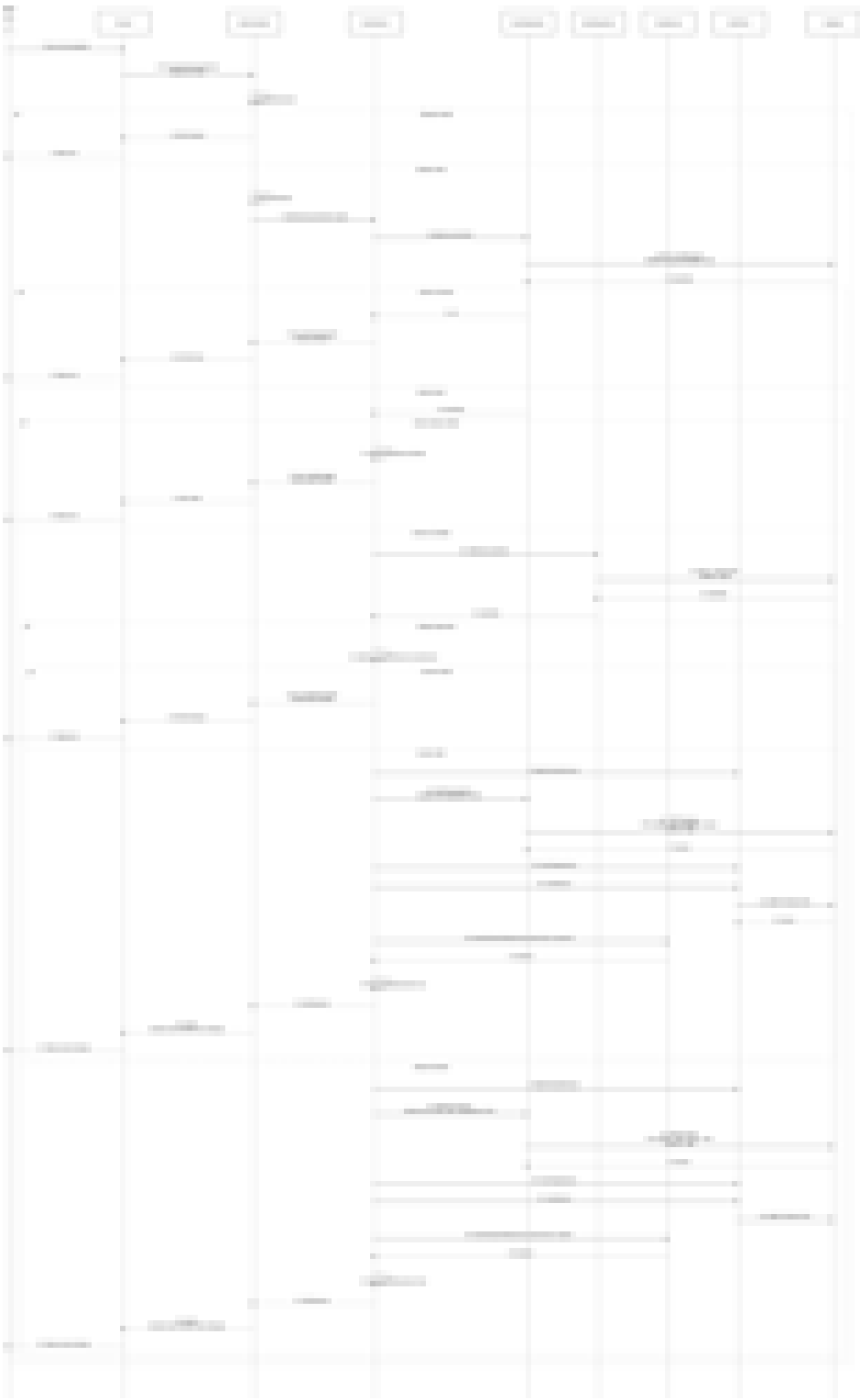
3.2.1.2 Sequence Diagram

3.2.2 Doctor Verification

3.2.2.1 Class Diagram



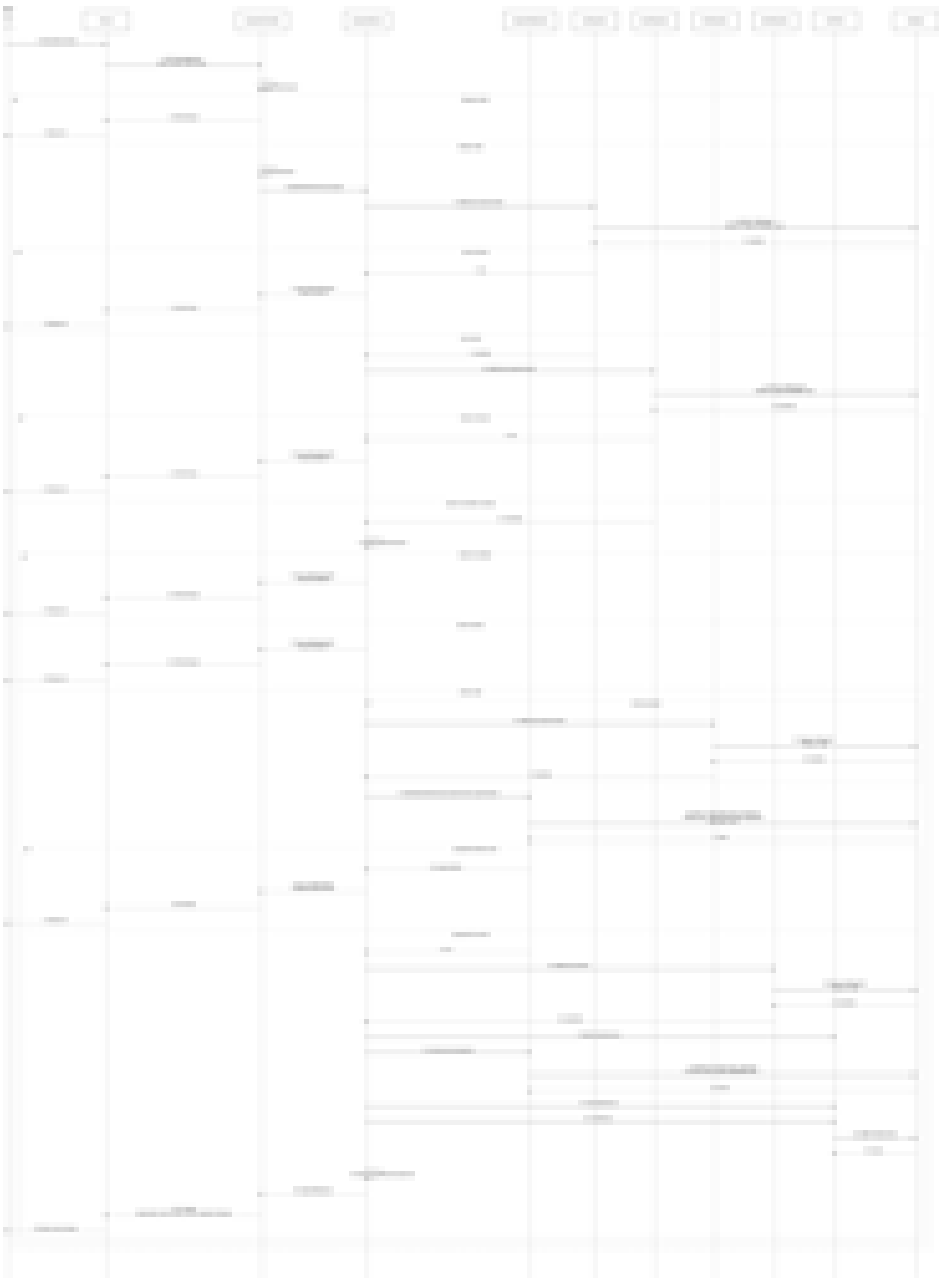
3.2.2.2 Sequence Diagram



3.2.3 Assign Patient to Doctor

3.2.3.1 Class Diagram

3.2.3.2 Sequence Diagram

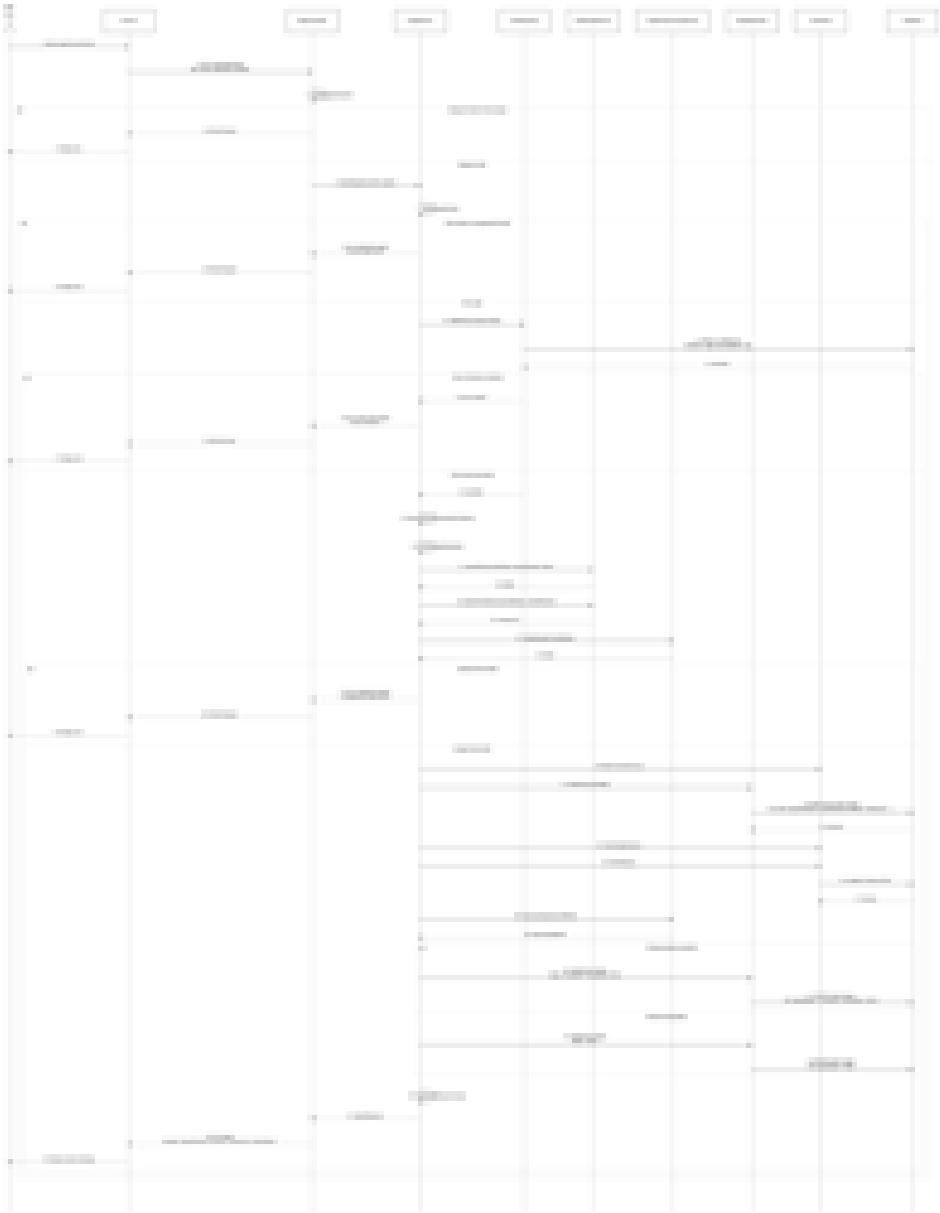


3.3 Retinal Image Management

3.3.1 Upload Image

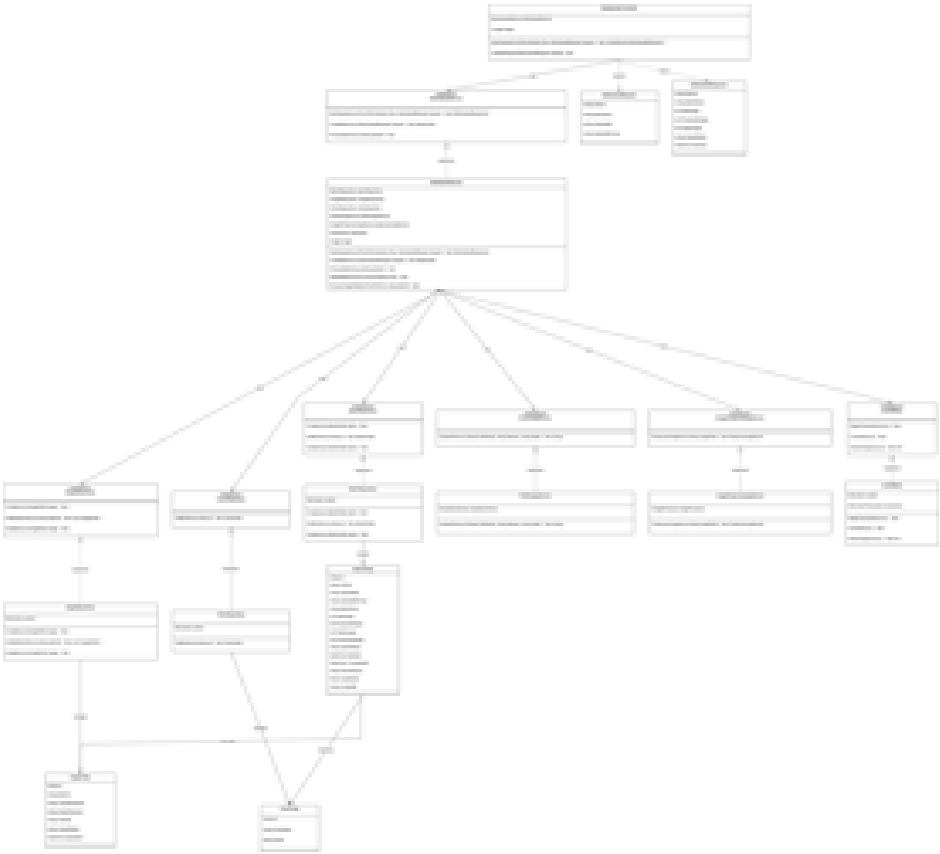
3.3.1.1 Class Diagram

3.3.1.2 Sequence Diagram



3.3.2 Bulk Upload

3.3.2.1 Class Diagram

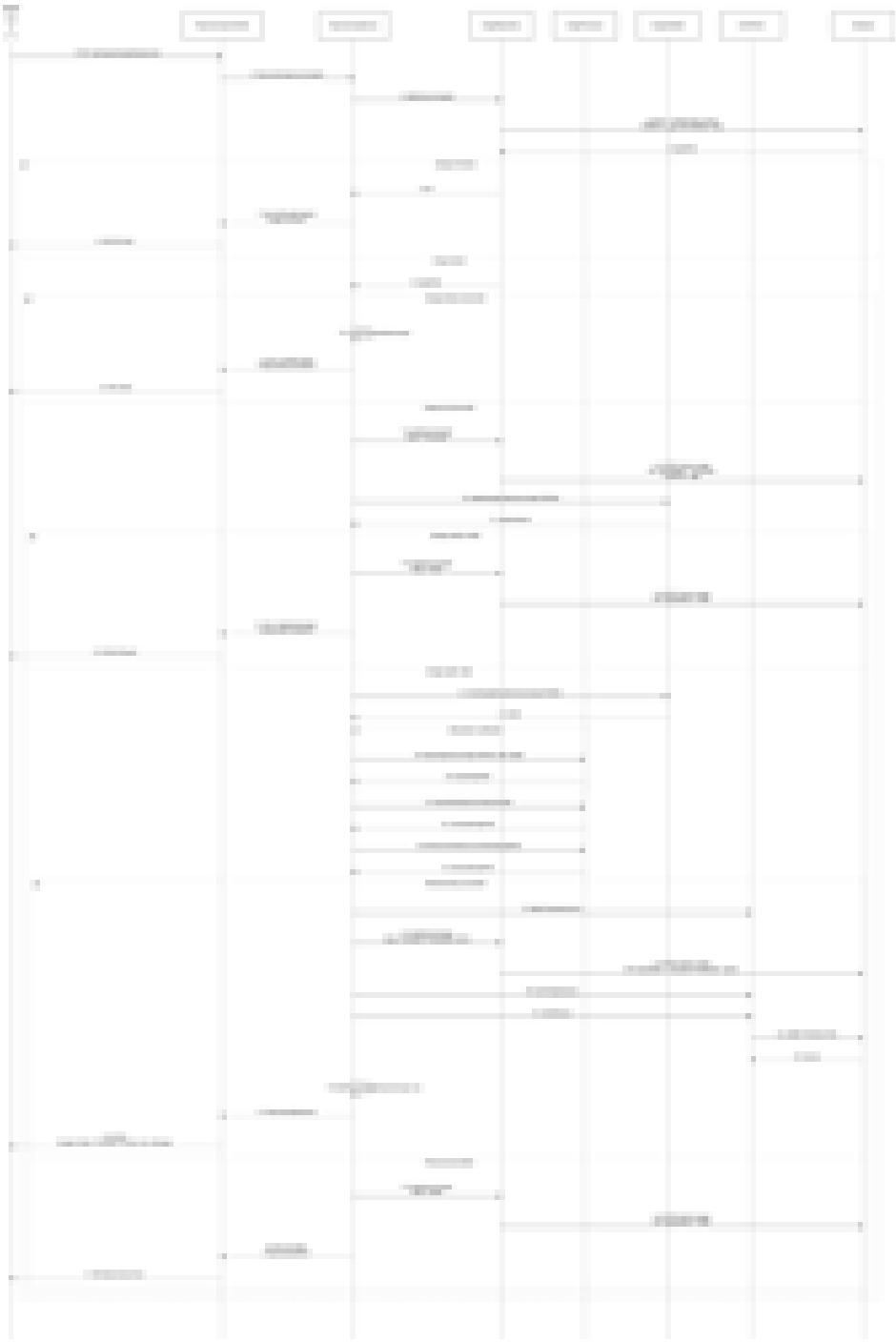


3.3.2.2 Sequence Diagram

3.3.3 Image Preprocessing

3.3.3.1 Class Diagram

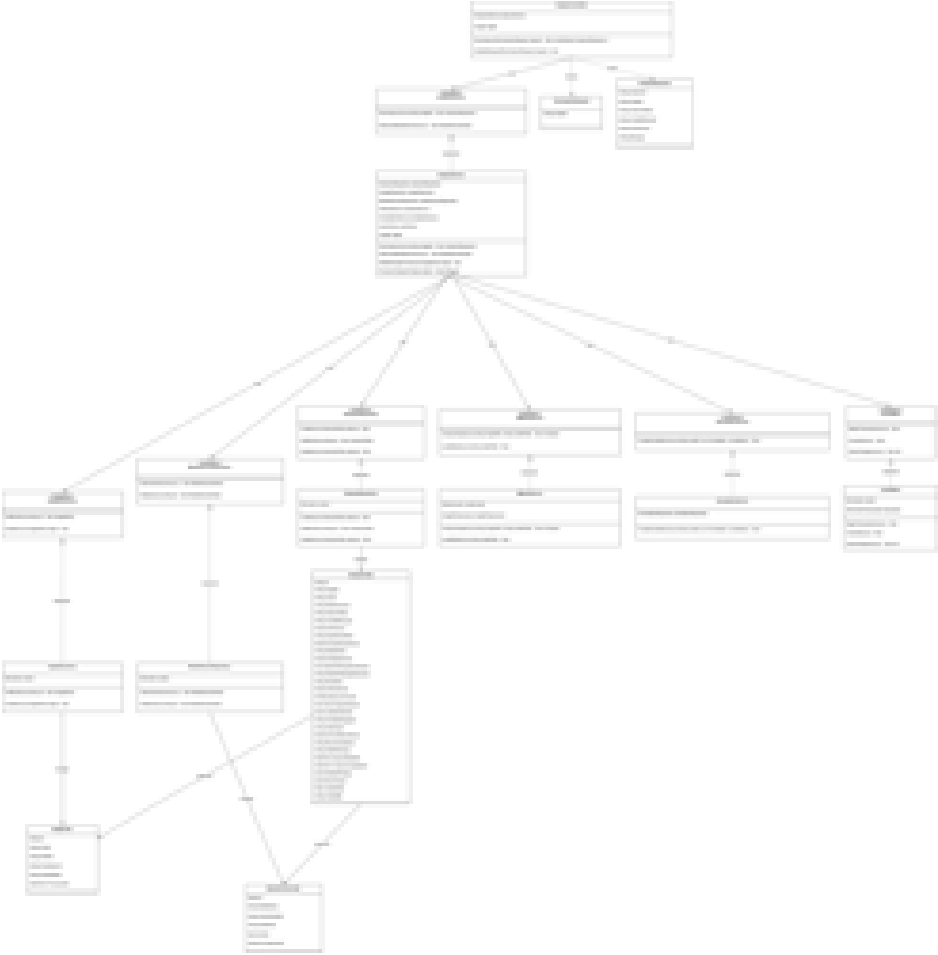
3.3.3.2 Sequence Diagram



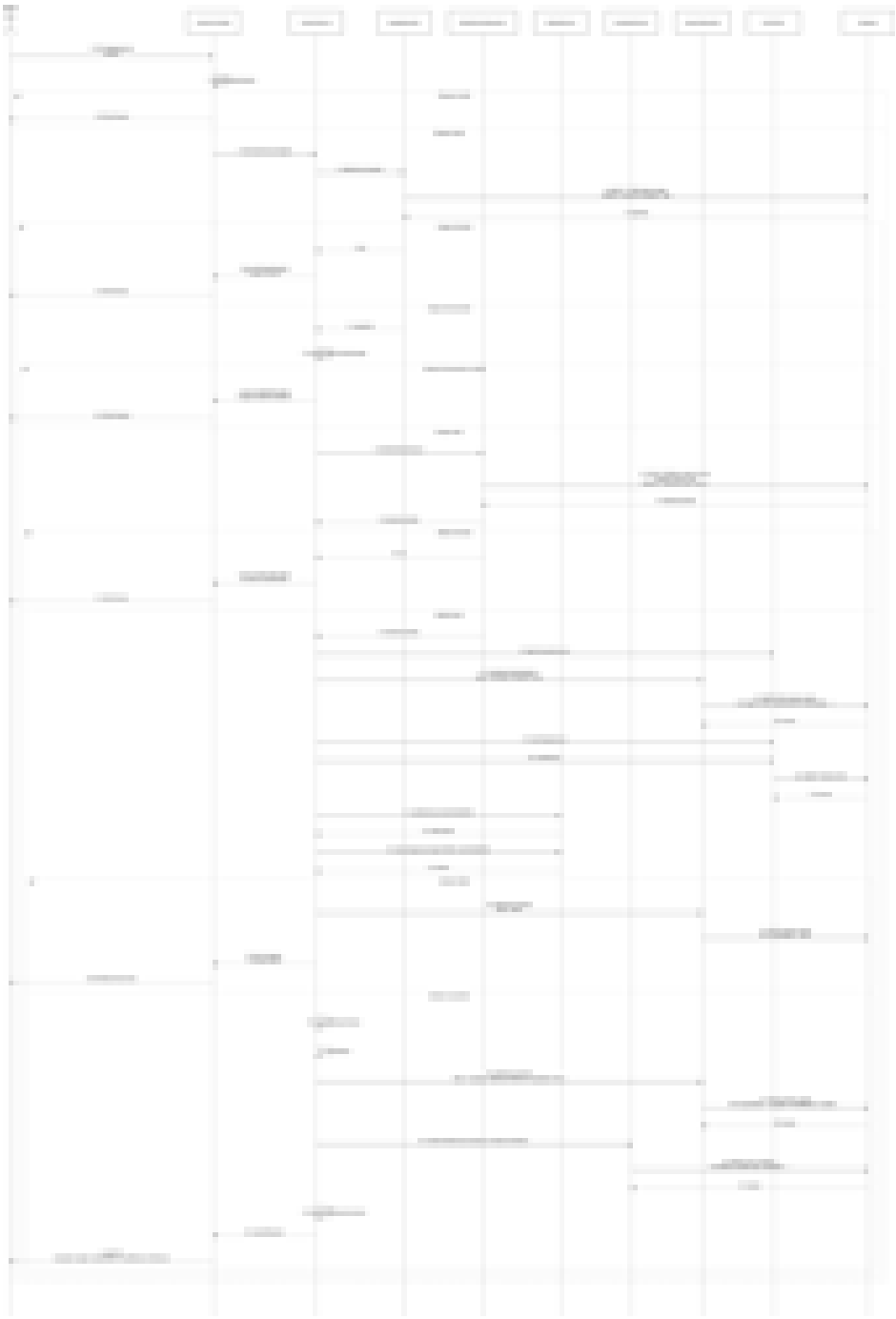
3.4 AI Analysis

3.4.1 Run AI Analysis

3.4.1.1 Class Diagram



3.4.1.2 Sequence Diagram



3.4.2 Store Analysis Result

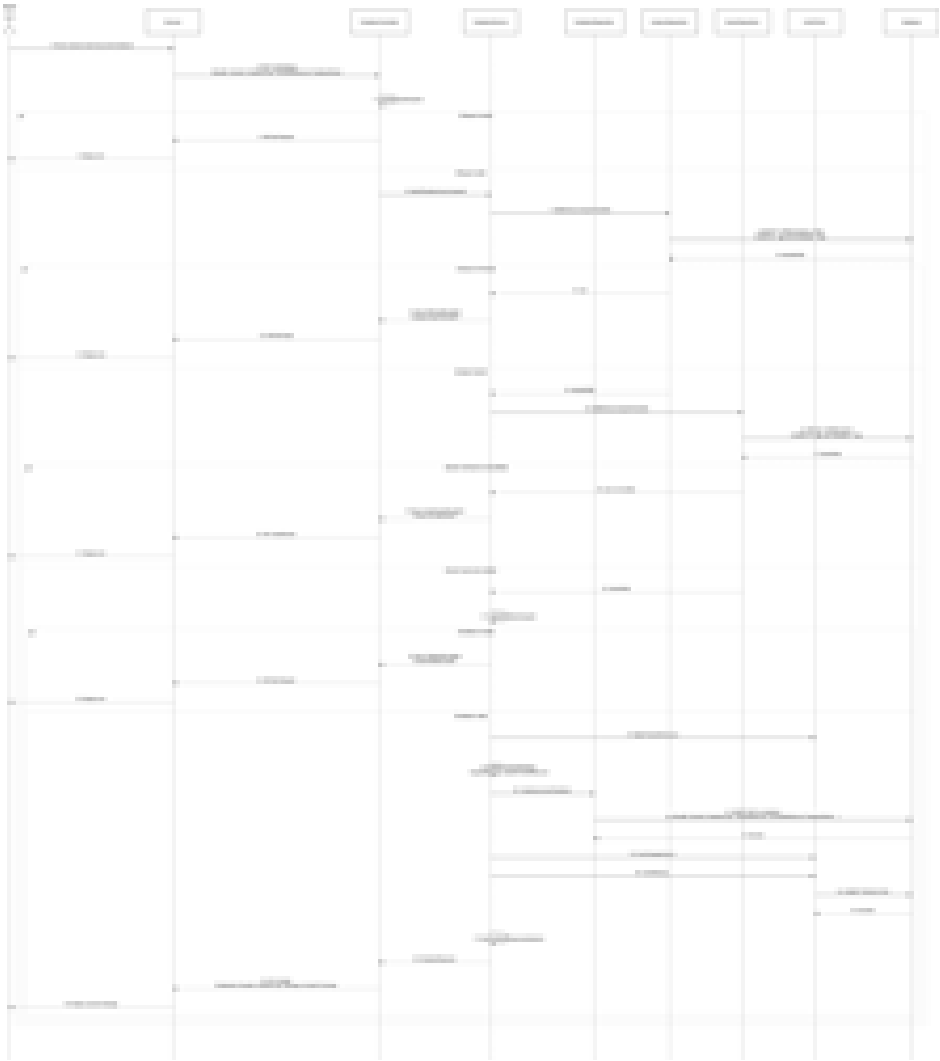
3.4.2.1 Class Diagram

3.4.2.2 Sequence Diagram

3.4.3 Doctor Feedback to AI

3.4.3.1 Class Diagram

3.4.3.2 Sequence Diagram

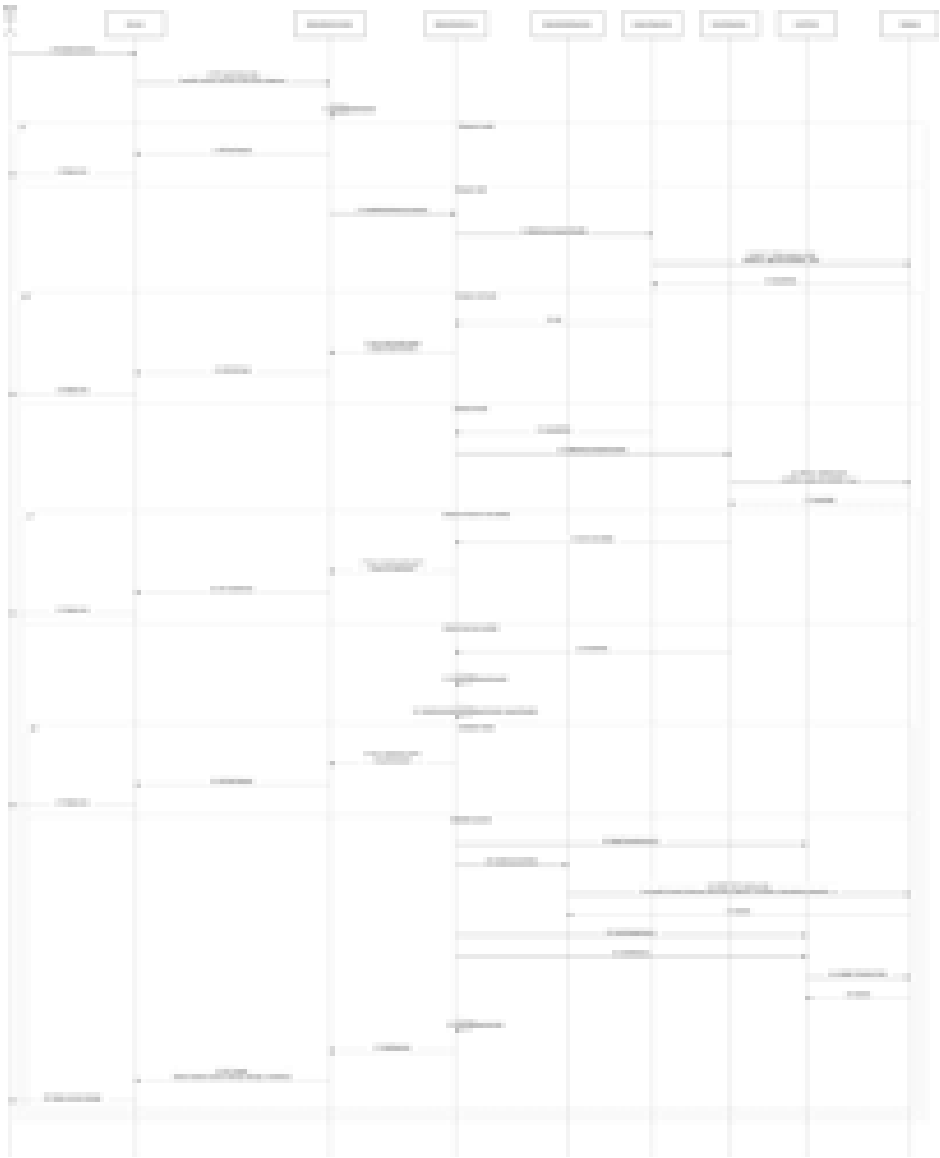


3.5 Medical Notes & Diagnosis

3.5.1 Create Medical Note

3.5.1.1 Class Diagram

3.5.1.2 Sequence Diagram



3.5.2 Update Diagnosis

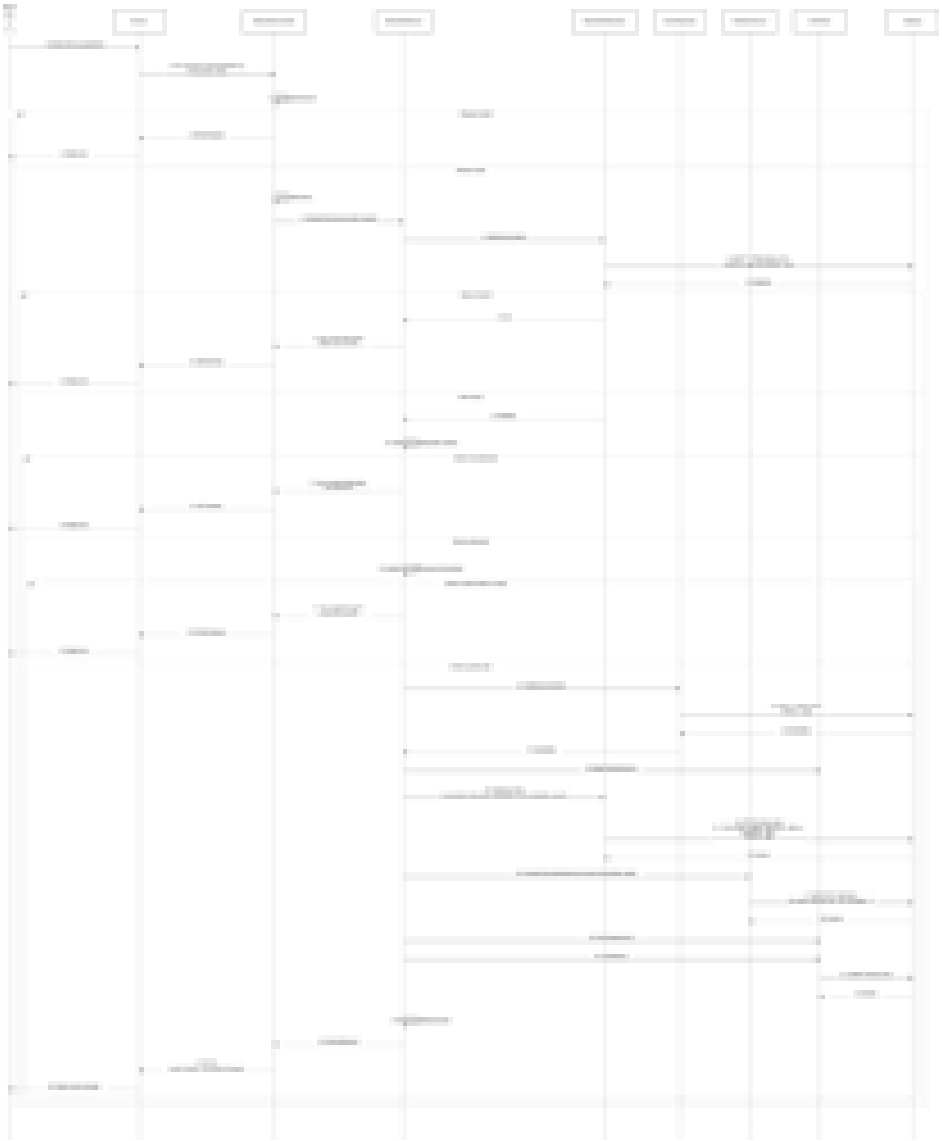
3.5.2.1 Class Diagram

3.5.2.2 Sequence Diagram

3.5.3 Follow-up Scheduling

3.5.3.1 Class Diagram

3.5.3.2 Sequence Diagram

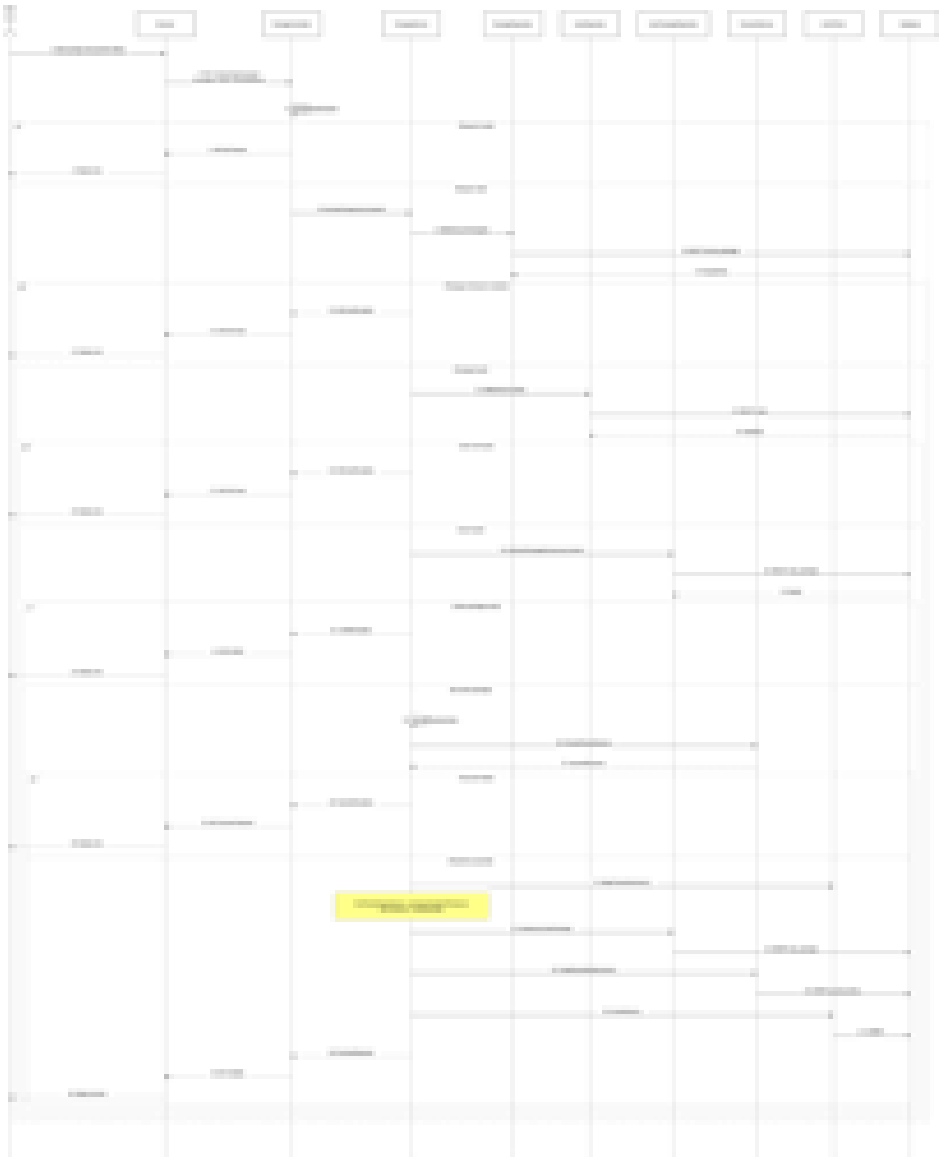


3.6 Package & Payment

3.6.1 Purchase Package

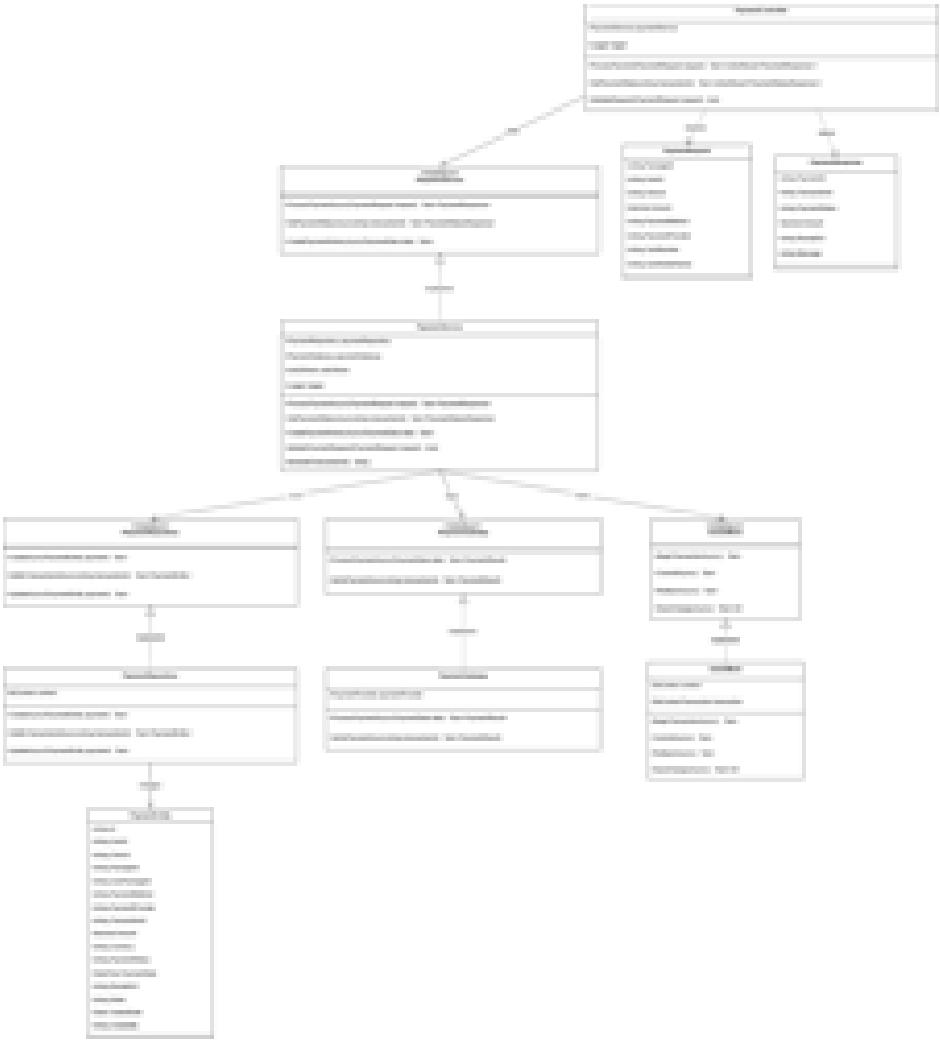
3.6.1.1 Class Diagram

3.6.1.2 Sequence Diagram

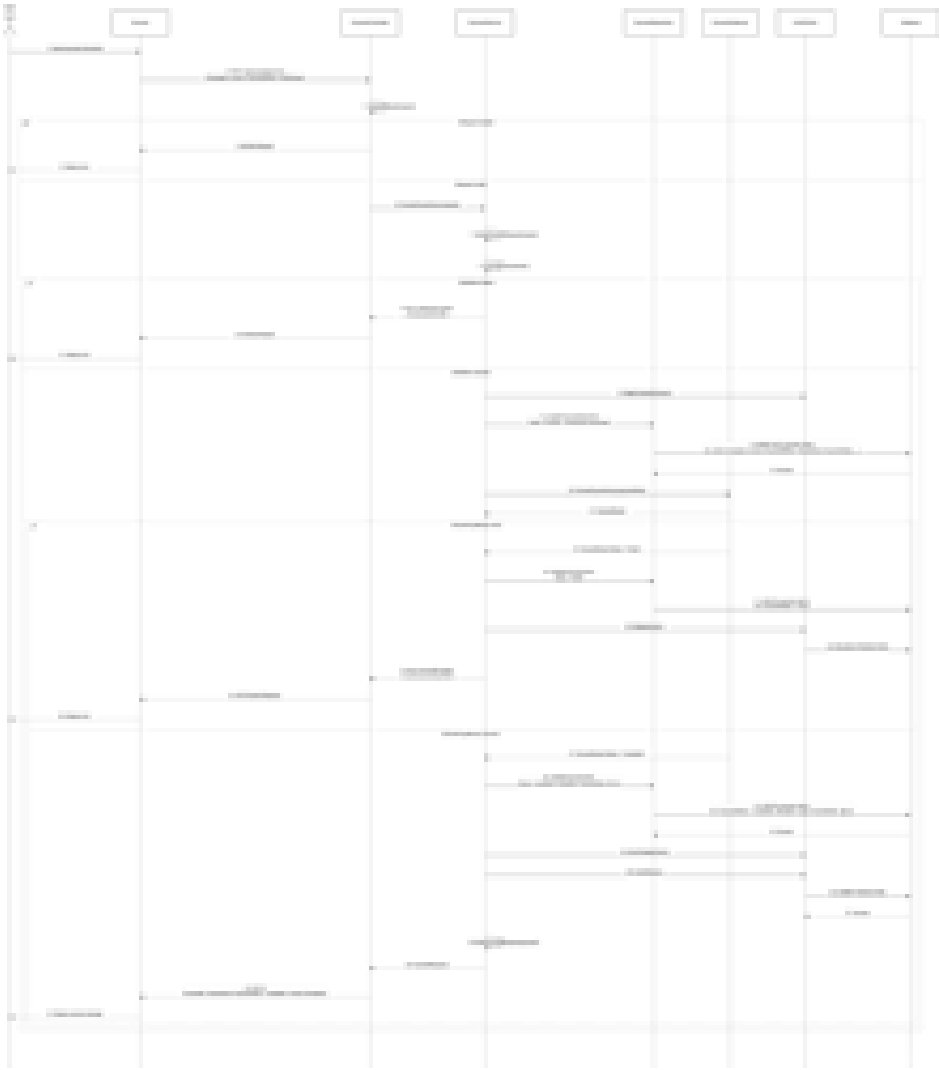


3.6.2 Payment Processing

3.6.2.1 Class Diagram

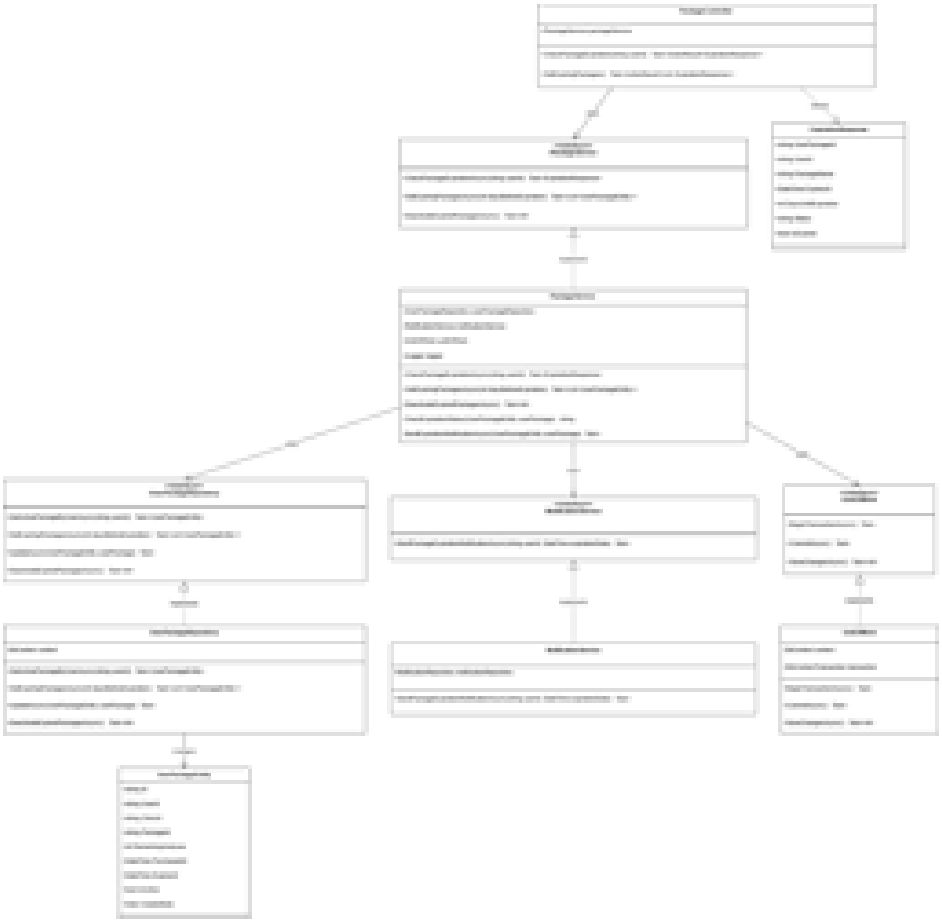


3.6.2.2 Sequence Diagram

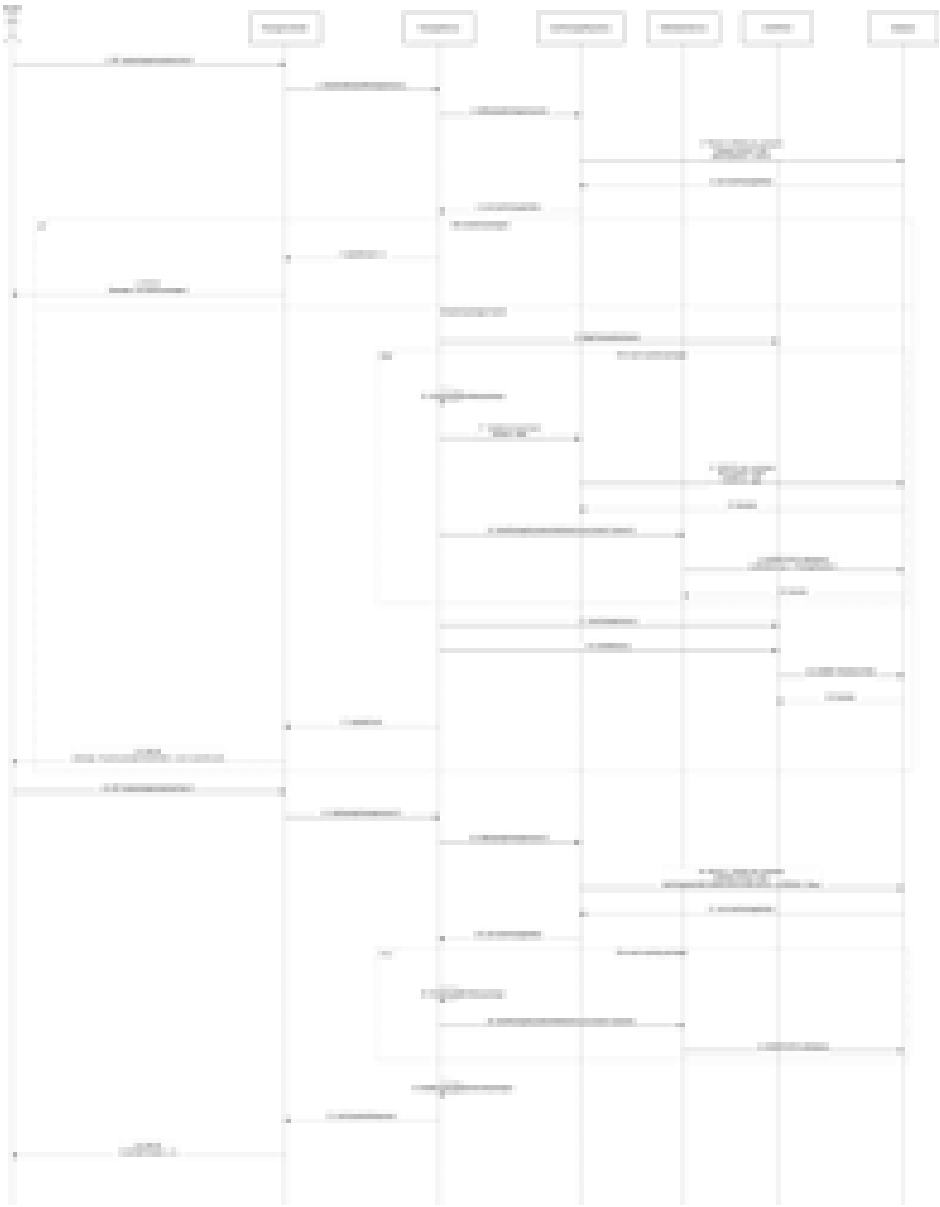


3.6.3 Package Expiration

3.6.3.1 Class Diagram



3.6.3.2 Sequence Diagram



3.7 Notification & Messaging

3.7.1 Send Notification

3.7.1.1 Class Diagram

3.7.1.2 Sequence Diagram

3.7.2 Receive Message

3.7.2.1 Class Diagram

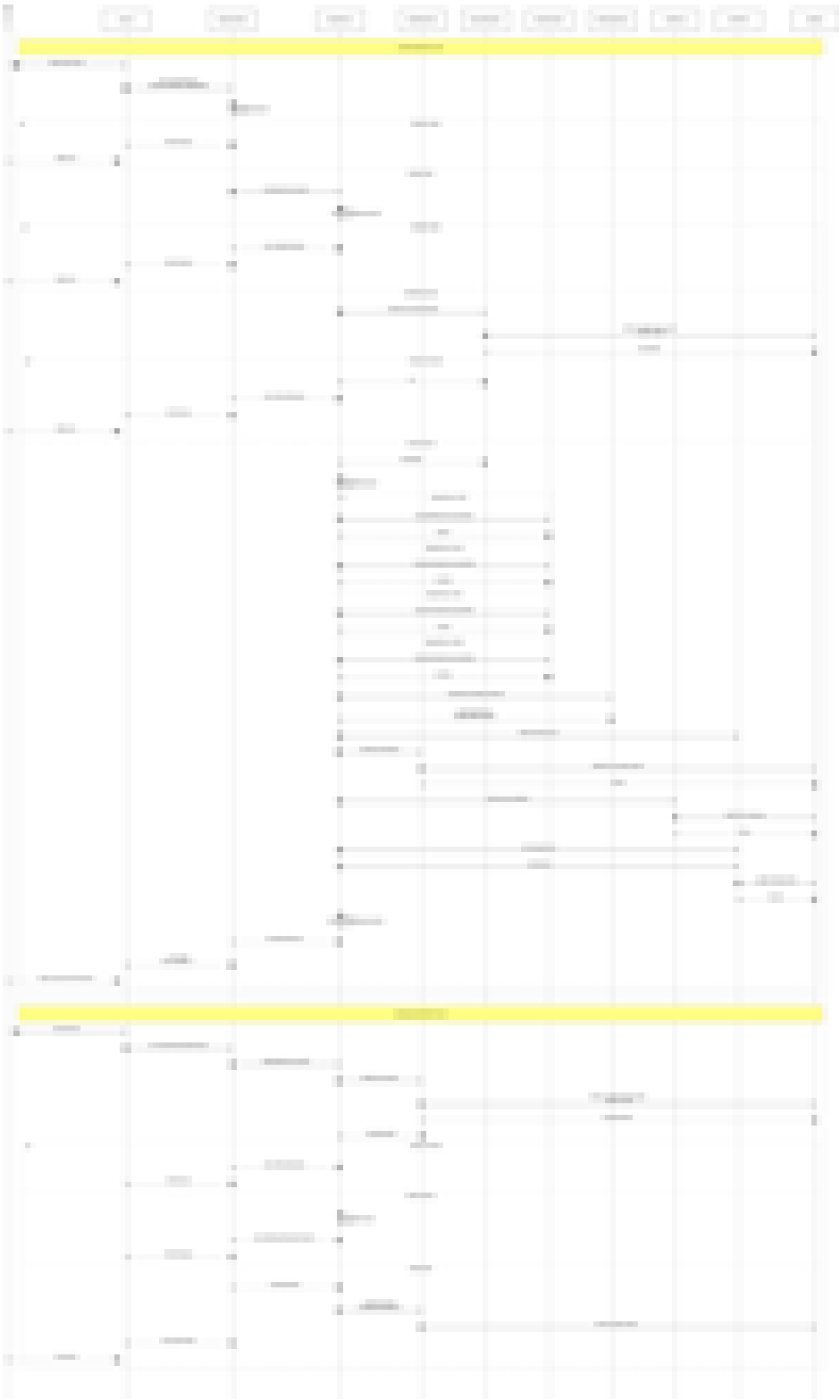
3.7.2.2 Sequence Diagram

3.8 Reporting & Audit

3.8.1 Export Report

3.8.1.1 Class Diagram

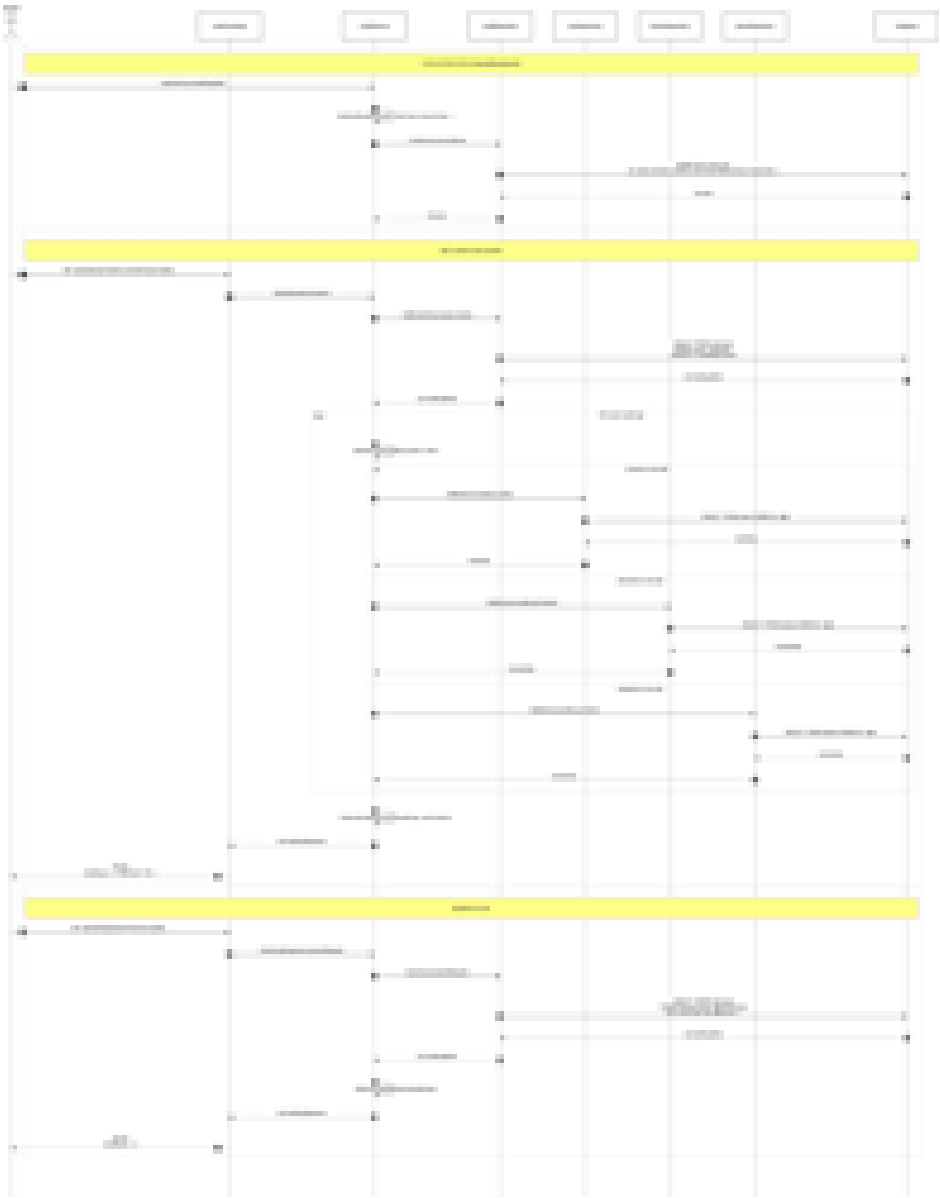
3.8.1.2 Sequence Diagram



3.8.2 System Audit Logging

3.8.2.1 Class Diagram

3.8.2.2 Sequence Diagram



V. SofSoftware Testing Documentation

This Software Testing Documentation outlines the scope, strategy, plan, and test cases for the System for Retinal Vascular Health Screening (AURA). The objective is to ensure the system operates correctly according to the Functional Requirements [FR-1 to FR-39] and Non-Functional Requirements [NFR-1 to NFR-23], with a specific focus on the accuracy of the AI model and the security of medical data.

1 Scope of Testing

The scope of testing covers all major components of the AURA system, ranging from the user interface to backend services and the AI Core.

Functional Modules:

+ **User (Patient) Module:** Registration/Login, Retinal image upload, Viewing AI analysis results, Profile management.

+ **Doctor Module:** Viewing patient lists, Validating AI results, Providing final diagnosis, Sending feedback.

+ **Clinic Module:** Managing doctors and patients, Bulk image upload, Viewing clinic statistical reports.

+ **Admin Module:** User/Clinic management, AI parameter configuration, System dashboard monitoring.

Technical Components:

+ **Web Application:** Frontend interface (React + TypeScript) ensuring compatibility and User Experience (UX).

+ **Backend API:** RESTful APIs (.NET) handling business logic, Authentication, and Data management.

+ **AI Core Microservice:** AI Service (Python) for image processing, vascular analysis, and result generation (Heatmap, Risk Score).

+ **Database:** Data integrity checks on PostgreSQL/MongoDB.

Non-Functional Requirements:

+ **Performance:** Verifying AI response time (< 20s) and page load time (< 3s).

+ **Security:** Verifying data encryption, Access Control (RBAC), and API security.

2 Test Strategy

The project adopts a testing process based on the Agile Scrum model with a "Prevention over Inspection" philosophy. Testing levels are distributed as follows:

2.1. Reviewing (Documentation & Design Check)

Objective: Detect errors early in the Software Requirement Specification (SRS) and system design.

Target: SRS Document, Use Cases, ERD, Wireframes.

Coverage: 100% of documentation and design artifacts.

2.2. Unit Test

Objective: Ensure individual code modules function correctly.

Backend (.NET): Focus on critical Services such as Authentication and Reporting (Target > 85%).

AI Core (Python): Verify image preprocessing functions (Target > 90%).

Frontend (React): Check Core Components (Upload, Dashboard) (Target > 70%).

2.3. Integration Test

Objective: Verify communication between system components.

Focus:

+ Communication between Web Server (.NET) and AI Service (Python).

+ Testing 100% of RESTful API endpoints.

+ Ensuring correct data flow from image upload to result storage in the Database.

2.4. System Test

Objective: Test the complete system as a finished product.

Scope: Verify 100% of Functional Requirements (FR-1 to FR-39).

Performance Testing: Load testing to ensure system stability during bulk image processing.

2.5. Acceptance Test (UAT)

Objective: Confirm the system meets the business needs of end-users (Doctors, Clinics).

Scenario: Execute key business flows ("Happy Path") and verify the Explainability of AI results for doctors.

3 Test Plan

The test plan is integrated into the project development phases:

Phase	Duration	Key Testing Activities	Person in Charge

Phase 1: Inception	Sprint 1	Review SRS, ERD, and system architecture. Set up the test environment.	Whole Team
Phase 2: Elaboration	Sprint 2-3	Unit Test for AI Core and basic Backend API. Integration Test for Upload and Auth APIs.	Backend Dev, AI Lead
Phase 3: Construction	Sprint 4-5	Integration Test for Web-AI flow. System Test for User/Doctor features. Security testing.	Tester, Frontend Dev
Phase 4: Transition	Sprint 6	Performance Tuning (AI < 20s). User Acceptance Testing (UAT). User Guide verification.	Whole Team, Supervisor

4 Test Cases

Below are typical Test Cases for the core features of AURA:

TC-01: Upload Retinal Image (User/Doctor Feature)

Pre-condition: User is logged in.

Steps:

+ Navigate to the "Upload Analysis" page.

+ Select an image file (JPG/PNG/DICOM format).

+ Click the "Analyze" button.

Expected Result:

+ System displays a progress bar.

+ Image is successfully sent to the server.

+ Message "Upload successful, analysis in progress" is displayed.

Abnormal Case: Uploading a non-image file or file > 10MB -> System displays error "Invalid file format/size".

TC-02: AI Analysis Processing (System Feature)

Pre-condition: Image uploaded successfully.

Steps:

+ System triggers the AI Core service.

+ AI performs vascular analysis.

+ Returns JSON result with Heatmap image URL

Expected Result:

+ Processing time is < 20 seconds.

+ Result includes: Risk Score, Heatmap (thermal image of abnormalities).

+ Data is saved to Database with "Completed" status.

TC-03: Doctor Validation (Doctor Feature)

Pre-condition: An AI analysis result is "Pending Review".

Steps:

+ Doctor opens the report details.

+ Views original image and AI Heatmap.

+ Enters professional comments and adjusts risk level (if AI is incorrect).

+ Clicks "Confirm & Send Report".

Expected Result:

+ Report status changes to "Validated".

+ Patient receives a notification of the result.

TC-04: Security - Data Access

Pre-condition: Logged in as a User (Patient A).

Steps:

+ Attempt to access the URL for Patient B's medical record (by changing the ID in the URL).

Expected Result:

+ System blocks access.

+ Displays "403 Forbidden" or "Access Denied" error.

5Test Reports

The Defect Management process will follow the established regulations:

Tracking Tool: GitHub Issues or **Jira**.

Severity Levels:

+ Critical: System crash, data loss, or severe medical security violation (Must be fixed immediately).

+ High: Major function failure (e.g., AI returns no result) (Must be fixed before next release).

+ Medium: UI or minor logic errors not affecting the main flow.

+ Low: Typographical errors, minor UI misalignment.

Report Content:

+ *Summary:* Brief description of the defect.

+ *Description:* Detailed explanation.

+ *Steps to Reproduce:* Sequence of actions to trigger the defect.

+ *Expected vs. Actual Result:* What should happen vs. what happened.

+ *Severity:* Level of impact.

+ *Assignee:* Person responsible for fixing.

VI: RELEASE PACKAGE & USER GUIDES

6.1. Deliverable Package

The AURA system (SP26SE025) is delivered as a comprehensive release package containing all necessary software components, configuration files, and documentation to ensure effective deployment and operation.

The deliverables are categorized as follows:

No.	Deliverable Item	Description
1	**Project Management & Tracking**	We use industry-standard tools to track progress and manage assets: • **Trello/Jira:** Used for backlog organization and task tracking. • **Google Drive:** Stores project schedules, meeting minutes, and management documents.
2	**Source Codes**	The source code is version-controlled on GitHub, divided into three core components: • **Backend API:** .NET-based services for business logic and data management. • **Frontend Web Application:** React/TypeScript interface for Users, Doctors, and Admins. • **AI Core Microservice:** Python-based service for retinal image analysis.
3	**Configuration & Deployment**	• **Docker & Docker Compose:** Containerization configuration files (Dockerfile, docker-compose.yml) to orchestrate the entire system stack. • **Database Scripts:** Initialization and migration scripts for PostgreSQL.
4	**Documentation**	• **SRS (Software Requirement Specification):** Detailed functional and non-functional requirements. • **SDD (Software Design Description):** System architecture and database design. • **Test Documentation:** Test plans, test cases, and reports. • **User Guides:** Role-based manuals for system operation.
5	**Final Report**	A comprehensive document synthesizing the entire capstone development lifecycle and project outcomes.

6.2. Installation Guides

This section provides detailed instructions for deploying the AURA system. The system is designed to run on standard server infrastructure or cloud environments using Docker for consistency.

6.2.1. System Requirements

To ensure optimal performance, the following hardware and software configurations are recommended,:

- **Operating System:** Windows 10/11, Linux (Ubuntu 20.04+), or macOS.
- **Hardware:**

◦ **CPU:** Intel Core i5 (Minimum) / Intel Core i7 (Recommended for AI processing).

◦ **RAM:** Minimum 16GB (Recommended 32GB to run all containers simultaneously).

◦ **Storage:** 512GB SSD or higher.

- **Software Prerequisites:**

◦ Git.

◦ Docker Desktop & Docker Compose.

◦ Node.js 20+ (For Frontend local dev).

◦ .NET SDK 8.0 (For Backend local dev).

◦ Python 3.9+ (For AI Core local dev).

◦ PostgreSQL 15+.

6.2.2. Installation Instructions

Administrators can deploy the system using individual component installation or via Docker (Recommended).

Method 1: Docker Deployment (Recommended)

This method ensures all services (Backend, Frontend, AI Core, Database) run in a unified environment.

Step 1: Clone the repositories.

git clone

cd AURA-System

Step 2: Configure Environment Variables. Create a .env file in the root directory and configure the necessary keys (Database connection strings, JWT Secrets, AI Model paths),.

Step 3: Build and Run. Open a terminal in the project root and execute:

docker-compose up --build -d

Step 4: Verify Deployment.

- **Frontend**
- **Backend API**
- **AI Service**

Method 2: Manual Installation (Component-wise)

A. Backend (.NET)

1. Navigate to the backend directory: cd AURA-Backend
2. Update appsettings.json with your PostgreSQL connection string.
3. Run database migrations:
4. Start the service:

B. AI Core Microservice (Python)

1. Navigate to the AI directory: cd AURA-AI-Core
2. Create a virtual environment:
3. Install dependencies:
4. Start the AI server:

C. Frontend (React/TypeScript)

1. Navigate to the frontend directory: cd AURA-Frontend
2. Install dependencies:
3. Start the development server:
4. Open the browser using the Local URL provided in the terminal.

6.3. User Manual

6.3.1. Overview by Role

The AURA system supports four primary user roles, each with specific workflows:

- **User (Patient):** Upload retinal images, request analysis, view AI diagnostic results, and track medical history.
- **Doctor:** View assigned patients, validate AI analysis results, add medical notes, and provide final recommendations,.
- **Clinic:** Manage doctor/patient accounts, perform bulk uploads for screening campaigns, and view aggregate reports.
- **Admin:** Configure system settings, manage user roles, audit logs, and monitor system performance.

6.3.2. Detailed Guidelines

1. **Login & Authentication**

- **Step 1:** Access the AURA homepage.
- **Step 2:** Click the **"Login"** button.
- **Step 3:** Select your role (Patient/Doctor/Clinic/Admin).
- **Step 4:** Enter your registered Email and Password, then click **"Sign In"**.

◦ (Place for Screenshot: Login Screen)

2. **Patient: Upload & View Analysis**

- **Step 1:** Navigate to the **"Dashboard"** and click **"Upload Image"**.
- **Step 2:** Select a retinal image file (.jpg, .png) from your device.
- **Step 3:** Click **"Analyze"**. The system will send the image to the AI Core for processing.
- **Step 4:** Once processing is complete, click **"View Results"**.
- **Step 5:** Review the output, which includes the AI risk assessment and the image with marked abnormalities.

◦ (Place for Screenshot: Upload & Result Interface)

3. **Doctor: Validate Diagnosis**

- **Step 1:** Log in and go to **"Patient List"** or **"Pending Reviews"**.
- **Step 2:** Select a patient record to open the detailed view.
- **Step 3:** Compare the original image with the AI-annotated version.
- **Step 4:** In the **"Doctor's Notes"** section, enter your clinical observations and final conclusion.
- **Step 5:** Click **"Validate & Send"** to finalize the report for the patient.

◦ (Place for Screenshot: Doctor’s Validation Dashboard)

4. **Clinic: Bulk Upload & Management**

- **Step 1:** Navigate to the **"Campaigns"** tab.
- **Step 2:** Click **"Bulk Upload"** to upload a batch of retinal images (e.g., from a screening event).
- **Step 3:** Monitor the status of batch processing via the progress bar.
- **Step 4:** Once finished, access the **"Reports"** section to view statistical summaries of the campaign.

◦ (Place for Screenshot: Clinic Bulk Upload Feature)

5. **Admin: User Management**

- **Step 1:** Navigate to **"User Management"**.
- **Step 2:** View the list of all registered users. Use the search bar to find specific accounts.
- **Step 3:** Click **"Edit"** to update user roles or **"Deactivate"** to suspend access if necessary.

◦ (Place for Screenshot: Admin User Management Panel)

6.4. Maintenance and Support

To ensure long-term reliability, the following maintenance procedures are recommended:

1. **Logging & Monitoring:** Administrators should regularly check system logs via the Admin Dashboard to identify errors or unauthorized access attempts.
2. **Data Backup:** Configure automated daily backups for the PostgreSQL database to prevent data loss.
3. **AI Model Updates:** The AI Core is designed as a microservice, allowing the technical team to update the machine learning models without disrupting the main application.
4. **System Updates:** Regularly pull the latest Docker images and restart containers to apply security patches and feature enhancements.